

Deep Learning Final Project

Deep Learning Regression for Stellar Age Estimation of Red Giant Stars

```
In [1]: import os
os.makedirs('plots', exist_ok=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Astronomy data access
from astroquery.vizier import Vizier

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

np.random.seed(42)
tf.random.set_seed(42)
```

```
/Users/nakulpandit/Desktop/Modules/deep learning/final Project/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
```

```
In [2]: # Loading APO-K2 stellar age catalog from Vizier
catalog_id = "J/AJ/167/208/table3"

vizier = Vizier(
    columns=[
        "EPIC", "GaiaEDR3", "Teff", "e_Teff", "logg", "e_logg",
        "[Fe/H]", "e_[Fe/H]", "[a/M]", "e_[a/M]", "[O/Fe]", "e_[O/Fe]",
        "Mstar", "e_Mstar", "Rstar", "e_Rstar", "logg-seis", "e_logg-seis",
        "numax", "e_numax", "deltanu", "e_deltanu",
        "Age", "e_Age", "E_Age", "Age-mode"
    ],
    row_limit=-1
)

tables = vizier.get_catalogs(catalog_id)
table = tables[0]
df = table.to_pandas()

print("Shape:", df.shape)
```

```
print(df.head())
print(df.columns.tolist())
```

Shape: (1912, 26)

	EPIC	GaiaEDR3	Teff	e_Teff	logg	e_logg	[Fe/H]	\
0	211837778	656372815030872192	4759	9	2.38	0.02	-0.306	
1	210915682	58475709154017280	4757	9	2.44	0.03	-0.335	
2	210352614	37590412768055040	4755	9	2.36	0.03	-0.556	
3	214710068	4073182748305313536	4643	10	2.18	0.03	-0.774	
4	212351370	3607531430651890048	4999	11	2.47	0.03	-0.461	

	e_[Fe/H]	[a/M]	e_[a/M]	...	logg-seis	e_logg-seis	numax	e_numax
0	0.008	0.063	0.006	...	2.45	0.01	35.000000	0.9
1	0.008	0.077	0.007	...	2.48	0.01	37.700001	0.5
2	0.008	0.123	0.008	...	2.44	0.01	34.400002	0.7
3	0.010	0.301	0.010	...	2.18	0.01	19.100000	0.5
4	0.008	0.104	0.007	...	2.55	0.01	42.599998	0.7

	deltanu	e_deltanu	Age	e_Age	E_Age	Age-mode
0	3.97	0.01	2.3	0.5	0.700000	2.2
1	4.37	0.12	3.9	1.9	3.800000	3.5
2	4.38	0.12	9.2	4.6	9.200000	8.4
3	2.91	0.14	15.4	11.4	42.900002	12.2
4	4.80	0.07	3.0	1.0	1.400000	2.8

[5 rows x 26 columns]

```
['EPIC', 'GaiaEDR3', 'Teff', 'e_Teff', 'logg', 'e_logg', '[Fe/H]', 'e_[Fe/H]', '[a/M]', 'e_[a/M]', '[O/Fe]', 'e_[O/Fe]', 'Mstar', 'e_Mstar', 'Rstar', 'e_Rstar', 'logg-seis', 'e_logg-seis', 'numax', 'e_numax', 'deltanu', 'e_deltanu', 'Age', 'e_Age', 'E_Age', 'Age-mode']
```

In [3]: *# Cleaning column names and inspecting the missing values*

```
df = df.rename(columns={
    "Teff": "teff",
    "e_Teff": "teff_err",
    "logg": "logg_spec",
    "e_logg": "logg_spec_err",
    "[Fe/H]": "feh",
    "e_[Fe/H]": "feh_err",
    "[a/M]": "alpha_m",
    "e_[a/M]": "alpha_m_err",
    "[O/Fe]": "o_fe",
    "e_[O/Fe]": "o_fe_err",
    "Mstar": "mass",
    "e_Mstar": "mass_err",
    "Rstar": "radius",
    "e_Rstar": "radius_err",
    "logg-seis": "logg_seis",
    "e_logg-seis": "logg_seis_err",
    "Age": "age_median",
    "e_Age": "age_err_low",
    "E_Age": "age_err_high",
    "Age-mode": "age_mode"
})
```

```
print(df.info())  
print("\nMissing values:\n", df.isnull().sum().sort_values(ascending=False))
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1912 entries, 0 to 1911
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   EPIC                   1912 non-null   int32
1   GaiaEDR3               1912 non-null   int64
2   teff                   1912 non-null   int16
3   teff_err               1912 non-null   int16
4   logg_spec              1912 non-null   float32
5   logg_spec_err          1912 non-null   float32
6   feh                    1912 non-null   float32
7   feh_err                1912 non-null   float32
8   alpha_m                1912 non-null   float32
9   alpha_m_err            1912 non-null   float32
10  o_fe                   1912 non-null   float32
11  o_fe_err               1912 non-null   float32
12  mass                   1912 non-null   float32
13  mass_err               1912 non-null   float32
14  radius                 1912 non-null   float32
15  radius_err             1912 non-null   float32
16  logg_seis              1912 non-null   float32
17  logg_seis_err          1912 non-null   float32
18  numax                  1912 non-null   float32
19  e_numax                1912 non-null   float32
20  deltanu                1912 non-null   float32
21  e_deltanu              1912 non-null   float32
22  age_median             1912 non-null   float32
23  age_err_low            1912 non-null   float32
24  age_err_high           1912 non-null   float32
25  age_mode               1912 non-null   float32
dtypes: float32(22), int16(2), int32(1), int64(1)
memory usage: 194.3 KB
None

```

```

Missing values:
EPIC          0
GaiaEDR3      0
age_err_high  0
age_err_low   0
age_median    0
e_deltanu     0
deltanu       0
e_numax       0
numax         0
logg_seis_err 0
logg_seis     0
radius_err    0
radius        0
mass_err      0
mass          0
o_fe_err      0
o_fe          0
alpha_m_err   0
alpha_m       0
feh_err       0

```

```
feh            0
logg_spec_err  0
logg_spec      0
teff_err       0
teff           0
age_mode       0
dtype: int64
```

In [4]: *# Defining Target*

```
# Selected input features
feature_cols = [
    "teff",
    "logg_spec",
    "feh",
    "alpha_m",
    "o_fe",
    "mass",
    "radius",
    "logg_seis",
    "numax",
    "deltanu"
]

target_col = "age_mode"
error_cols = [
    "teff_err", "logg_spec_err", "feh_err", "alpha_m_err", "o_fe_err",
    "mass_err", "radius_err", "logg_seis_err", "e_numax", "e_deltanu",
    "age_err_low", "age_err_high"
]

use_cols = feature_cols + [target_col] + error_cols

data = df[use_cols].copy()
print(data.head())
```

	teff	logg_spec	feh	alpha_m	o_fe	mass	radius	logg_seis	numa
x \									
0	4759	2.38	-0.306	0.063	0.105	1.49	12.00	2.45	35.00000
0									
1	4757	2.44	-0.335	0.077	0.159	1.26	10.65	2.48	37.70000
1									
2	4755	2.36	-0.556	0.123	0.259	0.95	9.68	2.44	34.40000
2									
3	4643	2.18	-0.774	0.301	0.369	0.81	12.05	2.18	19.10000
0									
4	4999	2.47	-0.461	0.104	0.201	1.34	10.20	2.55	42.59999
8									

	deltanu	...	feh_err	alpha_m_err	o_fe_err	mass_err	radius_err	\
0	3.97	...	0.008	0.006	0.018	0.12	0.31	
1	4.37	...	0.008	0.007	0.019	0.23	0.61	
2	4.38	...	0.008	0.008	0.020	0.17	0.56	
3	2.91	...	0.010	0.010	0.023	0.25	1.17	
4	4.80	...	0.008	0.007	0.027	0.15	0.35	

	logg_seis_err	e_numax	e_deltanu	age_err_low	age_err_high
0	0.01	0.9	0.01	0.5	0.700000
1	0.01	0.5	0.12	1.9	3.800000
2	0.01	0.7	0.12	4.6	9.200000
3	0.01	0.5	0.14	11.4	42.900002
4	0.01	0.7	0.07	1.0	1.400000

[5 rows x 23 columns]

```
In [5]: # Data cleaning and physically sensible filtering

data = data.dropna(subset=feature_cols + [target_col]).copy()

# Removeing obviously non-physical or extreme target values
# These filters are conservative for coursework robustness
data = data[(data["age_mode"] > 0.0) & (data["age_mode"] <= 20.0)]

# Remove rows with invalid seismic quantities
data = data[(data["numax"] > 0) & (data["deltanu"] > 0)]

# Removeing stars with very large age uncertainty if available
if "age_err_low" in data.columns and "age_err_high" in data.columns:
    data = data[(data["age_err_low"] < 5.0) & (data["age_err_high"] < 5.0)]

print("Cleaned shape:", data.shape)
print(data.describe())
```

Cleaned shape: (936, 23)

	teff	logg_spec	feh	alpha_m	o_fe	\
count	936.000000	936.000000	936.000000	936.000000	936.000000	
mean	4701.530983	2.369338	-0.304338	0.113724	0.164438	
std	204.546027	0.248243	0.259624	0.095616	0.140254	
min	4175.000000	1.520000	-0.999000	0.000000	-0.531000	
25%	4554.750000	2.210000	-0.479000	0.035000	0.057750	
50%	4695.500000	2.380000	-0.301000	0.082000	0.133000	
75%	4842.250000	2.520000	-0.127000	0.197250	0.264250	
max	5374.000000	3.270000	0.505000	0.387000	0.813000	

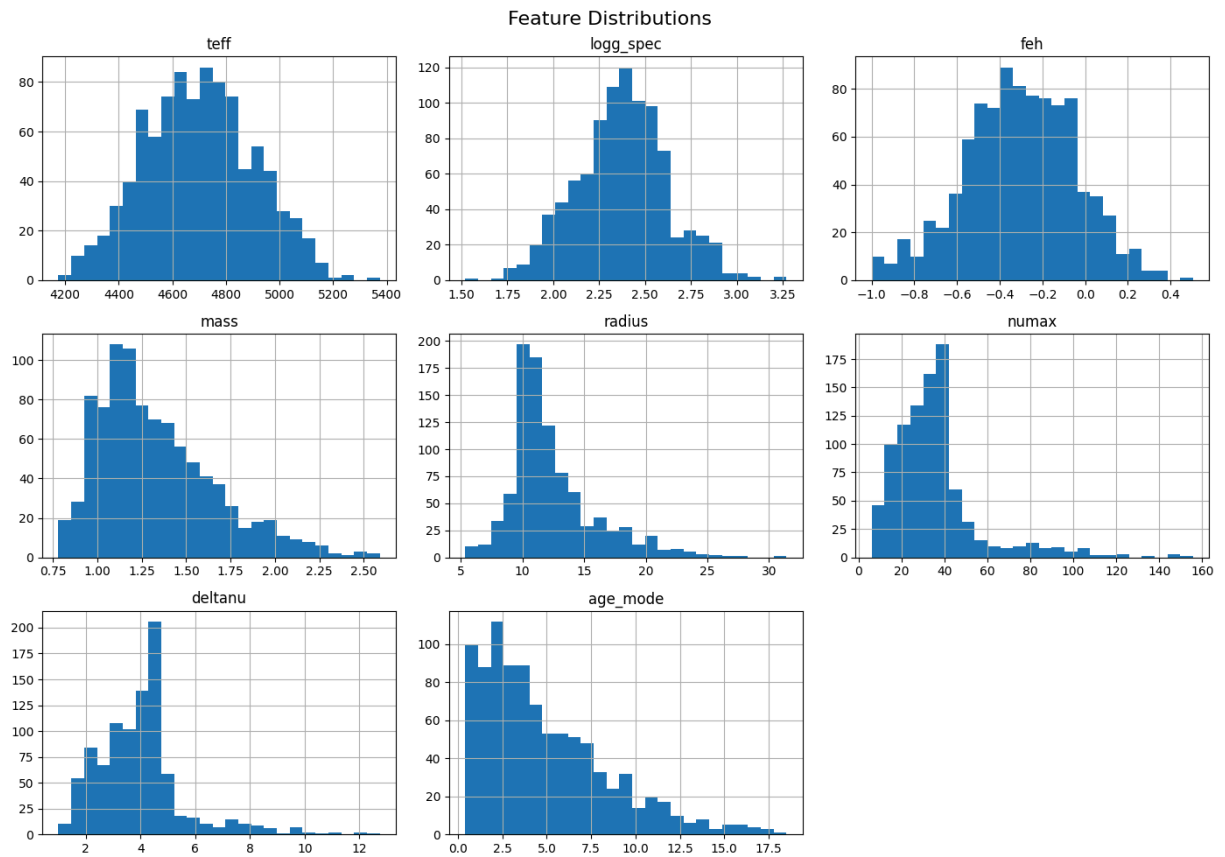
	mass	radius	logg_seis	numax	deltanu	...	\
count	936.000000	936.000000	936.000000	936.000000	936.000000	...	
mean	1.331389	12.411443	2.393066	35.397011	4.001624	...	
std	0.332136	3.538416	0.235654	20.814911	1.623964	...	
min	0.780000	5.350000	1.680000	6.100000	1.000000	...	
25%	1.080000	10.207500	2.250000	22.675001	2.980000	...	
50%	1.260000	11.365000	2.420000	32.950001	3.975000	...	
75%	1.520000	13.800000	2.510000	39.700001	4.590000	...	
max	2.590000	31.410000	3.120000	155.800003	12.740000	...	

	feh_err	alpha_m_err	o_fe_err	mass_err	radius_err	\
count	936.000000	936.000000	936.000000	936.000000	936.000000	
mean	0.009581	0.008764	0.022928	0.147468	0.449049	
std	0.003241	0.011295	0.013589	0.095551	0.322046	
min	0.006000	0.004000	0.008000	0.020000	0.050000	
25%	0.008000	0.006000	0.015000	0.090000	0.257500	
50%	0.008000	0.007000	0.019000	0.120000	0.370000	
75%	0.011000	0.009000	0.026000	0.180000	0.550000	
max	0.035000	0.284000	0.129000	0.920000	3.120000	

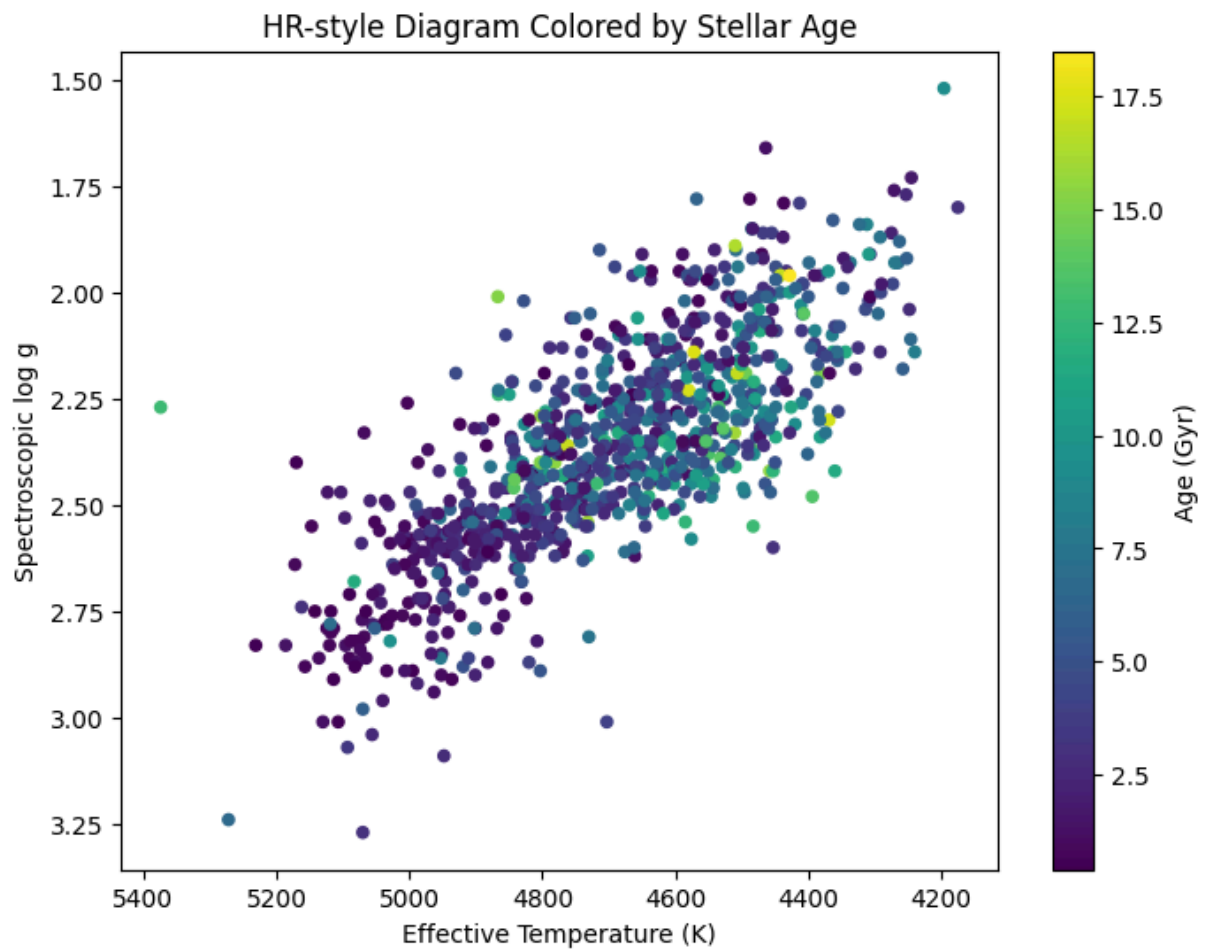
	logg_seis_err	e_numax	e_deltanu	age_err_low	age_err_high
count	936.000000	936.000000	936.000000	936.000000	936.000000
mean	0.008632	0.633333	0.049316	1.443056	2.183547
std	0.005891	0.392469	0.035735	0.884891	1.371750
min	0.000000	0.000000	0.000000	0.100000	0.100000
25%	0.010000	0.300000	0.020000	0.700000	1.000000
50%	0.010000	0.600000	0.040000	1.300000	2.000000
75%	0.010000	0.800000	0.070000	2.100000	3.300000
max	0.040000	2.000000	0.290000	3.800000	4.900000

[8 rows x 23 columns]

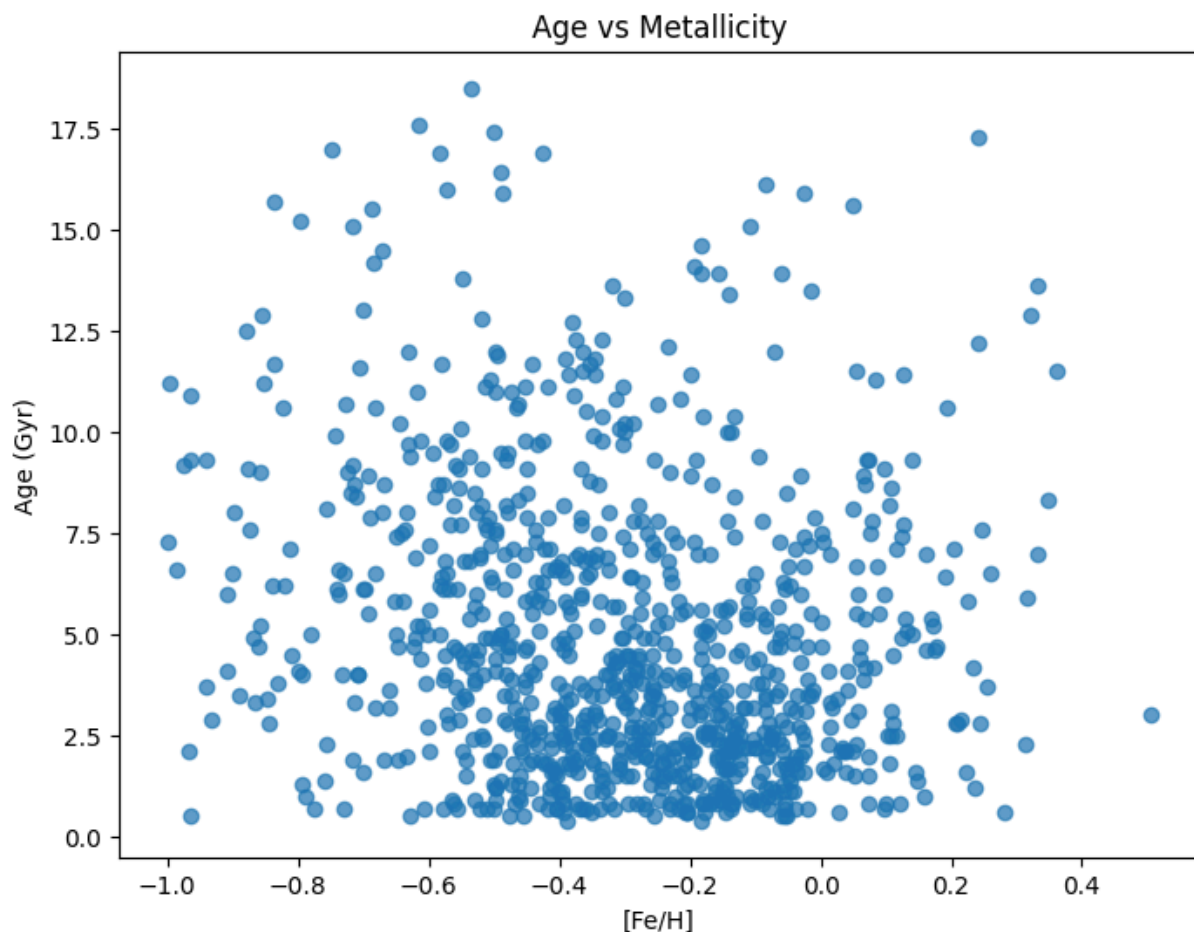
```
In [6]: # Histograms of important features
data[["teff", "logg_spec", "feh", "mass", "radius", "numax", "deltanu", "age_err_low", "age_err_high"]]
figsize=(14, 10), bins=25
)
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout()
plt.savefig('plots/feature_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [7]: # HR-style diagram: Teff vs logg (common stellar diagnostic view)
plt.figure(figsize=(8, 6))
plt.scatter(data["teff"], data["logg_spec"], c=data["age_mode"], s=20)
plt.gca().invert_xaxis() # astrophysics convention for temperature
plt.gca().invert_yaxis() # giant branch often shown with decreasing logg up
plt.xlabel("Effective Temperature (K)")
plt.ylabel("Spectroscopic log g")
plt.title("HR-style Diagram Colored by Stellar Age")
plt.colorbar(label="Age (Gyr)")
plt.savefig('plots/hr_style_diagram_colored_by_stellar_age.png', dpi=150, bt
plt.show()
```

```
In [8]: # Age vs metallicity
plt.figure(figsize=(8, 6))
plt.scatter(data["feh"], data["age_mode"], alpha=0.7)
plt.xlabel("[Fe/H]")
plt.ylabel("Age (Gyr)")
plt.title("Age vs Metallicity")
plt.savefig('plots/age_vs_metallicity.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [9]: X = data[feature_cols].copy()
y = data[target_col].copy()

# Train/temp split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Validation/test split
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)

print("Train shape:", X_train.shape)
print("Validation shape:", X_val.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (655, 10)
Validation shape: (140, 10)
Test shape: (141, 10)
```

```
In [10]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

```
In [11]: baseline_model = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(64, activation="relu"),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(1, activation="linear")
])

baseline_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="mse",
    metrics=["mae"]
)

baseline_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 128)	1
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 64)	8
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 32)	2
dense_3 (Dense)	(None, 1)	

Total params: 11,777 (46.00 KB)

Trainable params: 11,777 (46.00 KB)

Non-trainable params: 0 (0.00 B)

```
In [12]: early_stop = EarlyStopping(
    monitor="val_loss",
    patience=30,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=10,
    min_lr=1e-6,
    verbose=1
)

history = baseline_model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
```

```

epochs=300,
batch_size=32,
callbacks=[early_stop, reduce_lr],
verbose=1
)

```

Epoch 1/300

1/21  12s 647ms/step - loss: 30.2344 - mae: 4.6320

21/21  1s 4ms/step - loss: 29.0123 - mae: 4.1169 - val_loss: 19.1125 - val_mae: 3.2549 - learning_rate: 0.0010

Epoch 2/300

1/21  0s 8ms/step - loss: 12.7704 - mae: 2.9129

21/21  0s 1ms/step - loss: 13.2922 - mae: 2.6856 - val_loss: 8.6851 - val_mae: 2.1874 - learning_rate: 0.0010

Epoch 3/300

1/21  0s 5ms/step - loss: 3.1856 - mae: 1.4564

21/21  0s 1ms/step - loss: 6.1259 - mae: 1.7329 - val_loss: 4.1283 - val_mae: 1.3519 - learning_rate: 0.0010

Epoch 4/300

1/21  0s 5ms/step - loss: 1.4357 - mae: 0.9467

21/21  0s 1ms/step - loss: 3.2843 - mae: 1.1868 - val_loss: 2.4027 - val_mae: 0.9942 - learning_rate: 0.0010

Epoch 5/300

1/21  0s 5ms/step - loss: 1.1648 - mae: 0.8096

21/21  0s 1ms/step - loss: 2.5398 - mae: 1.0442 - val_loss: 1.7881 - val_mae: 0.8484 - learning_rate: 0.0010

Epoch 6/300

1/21  0s 5ms/step - loss: 1.7280 - mae: 1.0198

21/21  0s 1ms/step - loss: 2.2036 - mae: 0.9425 - val_loss: 1.3950 - val_mae: 0.7527 - learning_rate: 0.0010

Epoch 7/300

1/21  0s 5ms/step - loss: 1.6221 - mae: 0.9786

21/21  0s 1ms/step - loss: 1.8679 - mae: 0.9168 - val_loss: 1.2408 - val_mae: 0.7238 - learning_rate: 0.0010

Epoch 8/300

1/21  0s 5ms/step - loss: 1.7924 - mae: 1.0349

21/21  0s 1ms/step - loss: 1.7355 - mae: 0.8859 - val_loss: 1.1564 - val_mae: 0.7094 - learning_rate: 0.0010

Epoch 9/300

1/21  0s 5ms/step - loss: 2.0743 - mae: 1.0858

21/21  0s 1ms/step - loss: 1.8002 - mae: 0.8873 - val_loss: 1.1025 - val_mae: 0.6748 - learning_rate: 0.0010

Epoch 10/300

1/21  0s 5ms/step - loss: 0.8089 - mae: 0.6736

21/21  0s 1ms/step - loss: 1.5501 - mae: 0.8050 - val_loss: 1.0903 - val_mae: 0.6636 - learning_rate: 0.0010

Epoch 11/300
1/21  0s 5ms/step - loss: 0.8311 - mae: 0.7406

21/21  0s 1ms/step - loss: 1.4686 - mae: 0.7869 - val_loss: 1.1752 - val_mae: 0.6589 - learning_rate: 0.0010

Epoch 12/300
1/21  0s 5ms/step - loss: 0.9773 - mae: 0.7695

21/21  0s 1ms/step - loss: 1.4239 - mae: 0.8163 - val_loss: 1.0007 - val_mae: 0.6481 - learning_rate: 0.0010

Epoch 13/300
1/21  0s 5ms/step - loss: 0.7284 - mae: 0.6499

21/21  0s 997us/step - loss: 1.4041 - mae: 0.7919 - val_loss: 1.1132 - val_mae: 0.6599 - learning_rate: 0.0010

Epoch 14/300
1/21  0s 5ms/step - loss: 0.7466 - mae: 0.6977

21/21  0s 1ms/step - loss: 1.3010 - mae: 0.7624 - val_loss: 1.1565 - val_mae: 0.6816 - learning_rate: 0.0010

Epoch 15/300
1/21  0s 5ms/step - loss: 0.6511 - mae: 0.6381

21/21  0s 1ms/step - loss: 1.1778 - mae: 0.7528 - val_loss: 1.0168 - val_mae: 0.6325 - learning_rate: 0.0010

Epoch 16/300
1/21  0s 5ms/step - loss: 0.7629 - mae: 0.6036

21/21  0s 1ms/step - loss: 1.2461 - mae: 0.7158 - val_loss: 0.9812 - val_mae: 0.6437 - learning_rate: 0.0010

Epoch 17/300
1/21  0s 5ms/step - loss: 1.2672 - mae: 0.7976

21/21  0s 1ms/step - loss: 1.3339 - mae: 0.7493 - val_loss: 0.8888 - val_mae: 0.5969 - learning_rate: 0.0010

Epoch 18/300
1/21  0s 5ms/step - loss: 0.6250 - mae: 0.6005

21/21  0s 1ms/step - loss: 1.1958 - mae: 0.7132 - val_loss: 0.7842 - val_mae: 0.5865 - learning_rate: 0.0010

Epoch 19/300
1/21  0s 5ms/step - loss: 0.7807 - mae: 0.6830

21/21  0s 1ms/step - loss: 1.0185 - mae: 0.7062 - val_loss: 0.6588 - val_mae: 0.5471 - learning_rate: 0.0010

Epoch 20/300
1/21  0s 5ms/step - loss: 0.7969 - mae: 0.6097

21/21  0s 1ms/step - loss: 0.9836 - mae: 0.6654 - val_loss: 0.7058 - val_mae: 0.5460 - learning_rate: 0.0010

Epoch 21/300
1/21  0s 5ms/step - loss: 1.0033 - mae: 0.7046

21/21  0s 1ms/step - loss: 1.1108 - mae: 0.7116 - val_loss: 0.6074 - val_mae: 0.5259 - learning_rate: 0.0010

Epoch 22/300

1/21 ————— 0s 5ms/step - loss: 0.8552 - mae: 0.7277

21/21 ————— 0s 1ms/step - loss: 1.0661 - mae: 0.7161 - val_loss: 0.6658 - val_mae: 0.5347 - learning_rate: 0.0010
Epoch 23/300

1/21 ————— 0s 5ms/step - loss: 1.3792 - mae: 0.8504

21/21 ————— 0s 1ms/step - loss: 1.1248 - mae: 0.7134 - val_loss: 0.6404 - val_mae: 0.5307 - learning_rate: 0.0010
Epoch 24/300

1/21 ————— 0s 5ms/step - loss: 0.5364 - mae: 0.5361

21/21 ————— 0s 1ms/step - loss: 0.7743 - mae: 0.6229 - val_loss: 0.8394 - val_mae: 0.6002 - learning_rate: 0.0010
Epoch 25/300

1/21 ————— 0s 5ms/step - loss: 1.2014 - mae: 0.7568

21/21 ————— 0s 1ms/step - loss: 1.0314 - mae: 0.6894 - val_loss: 0.8918 - val_mae: 0.6087 - learning_rate: 0.0010
Epoch 26/300

1/21 ————— 0s 5ms/step - loss: 0.7659 - mae: 0.6674

21/21 ————— 0s 1ms/step - loss: 1.0463 - mae: 0.6564 - val_loss: 0.5569 - val_mae: 0.5124 - learning_rate: 0.0010
Epoch 27/300

1/21 ————— 0s 5ms/step - loss: 0.5425 - mae: 0.5420

21/21 ————— 0s 1ms/step - loss: 0.8347 - mae: 0.6080 - val_loss: 0.5797 - val_mae: 0.5012 - learning_rate: 0.0010
Epoch 28/300

1/21 ————— 0s 5ms/step - loss: 0.4979 - mae: 0.5296

21/21 ————— 0s 1ms/step - loss: 0.7995 - mae: 0.5979 - val_loss: 0.5602 - val_mae: 0.5054 - learning_rate: 0.0010
Epoch 29/300

1/21 ————— 0s 5ms/step - loss: 0.6106 - mae: 0.5316

21/21 ————— 0s 1000us/step - loss: 0.9189 - mae: 0.6253 - val_loss: 0.5514 - val_mae: 0.4990 - learning_rate: 0.0010
Epoch 30/300

1/21 ————— 0s 5ms/step - loss: 0.6960 - mae: 0.6221

21/21 ————— 0s 1ms/step - loss: 0.8021 - mae: 0.6359 - val_loss: 0.4846 - val_mae: 0.4649 - learning_rate: 0.0010
Epoch 31/300

1/21 ————— 0s 5ms/step - loss: 0.5692 - mae: 0.5532

21/21 ————— 0s 1ms/step - loss: 0.7820 - mae: 0.6017 - val_loss: 0.6031 - val_mae: 0.5334 - learning_rate: 0.0010
Epoch 32/300

1/21 ————— 0s 5ms/step - loss: 0.6599 - mae: 0.5907

21/21 ————— 0s 1ms/step - loss: 0.7661 - mae: 0.5746 - val_loss: 0.5095 - val_mae: 0.4681 - learning_rate: 0.0010
Epoch 33/300

1/21 ————— 0s 5ms/step - loss: 0.4761 - mae: 0.5330

21/21  0s 1ms/step - loss: 0.6686 - mae: 0.5448 - val_loss: 0.4907 - val_mae: 0.4788 - learning_rate: 0.0010
Epoch 34/300
1/21  0s 5ms/step - loss: 0.9978 - mae: 0.7266

21/21  0s 1ms/step - loss: 0.7625 - mae: 0.5944 - val_loss: 0.4098 - val_mae: 0.4379 - learning_rate: 0.0010
Epoch 35/300
1/21  0s 5ms/step - loss: 0.8857 - mae: 0.7657

21/21  0s 1ms/step - loss: 0.8888 - mae: 0.6465 - val_loss: 0.3457 - val_mae: 0.4054 - learning_rate: 0.0010
Epoch 36/300
1/21  0s 5ms/step - loss: 0.6273 - mae: 0.6028

21/21  0s 1ms/step - loss: 0.8804 - mae: 0.6138 - val_loss: 0.3309 - val_mae: 0.4120 - learning_rate: 0.0010
Epoch 37/300
1/21  0s 5ms/step - loss: 0.5457 - mae: 0.5502

21/21  0s 1ms/step - loss: 0.7576 - mae: 0.5841 - val_loss: 0.3793 - val_mae: 0.4344 - learning_rate: 0.0010
Epoch 38/300
1/21  0s 5ms/step - loss: 0.4795 - mae: 0.4934

21/21  0s 1ms/step - loss: 0.6463 - mae: 0.5706 - val_loss: 0.7606 - val_mae: 0.6067 - learning_rate: 0.0010
Epoch 39/300
1/21  0s 5ms/step - loss: 0.6386 - mae: 0.5907

21/21  0s 1ms/step - loss: 0.6278 - mae: 0.5486 - val_loss: 0.7314 - val_mae: 0.6106 - learning_rate: 0.0010
Epoch 40/300
1/21  0s 5ms/step - loss: 0.7458 - mae: 0.6754

21/21  0s 1ms/step - loss: 0.6864 - mae: 0.5748 - val_loss: 0.6735 - val_mae: 0.5873 - learning_rate: 0.0010
Epoch 41/300
1/21  0s 5ms/step - loss: 1.0468 - mae: 0.5085

21/21  0s 1ms/step - loss: 0.7776 - mae: 0.5408 - val_loss: 0.8321 - val_mae: 0.6082 - learning_rate: 0.0010
Epoch 42/300
1/21  0s 5ms/step - loss: 0.7141 - mae: 0.6307

21/21  0s 1ms/step - loss: 0.6624 - mae: 0.5585 - val_loss: 0.5008 - val_mae: 0.4992 - learning_rate: 0.0010
Epoch 43/300
1/21  0s 5ms/step - loss: 0.6461 - mae: 0.5657

21/21  0s 1ms/step - loss: 0.6317 - mae: 0.5105 - val_loss: 0.4610 - val_mae: 0.4458 - learning_rate: 0.0010
Epoch 44/300
1/21  0s 5ms/step - loss: 0.5720 - mae: 0.5222

21/21  0s 1ms/step - loss: 0.7565 - mae: 0.5675 - val_loss: 0.5240 - val_mae: 0.4960 - learning_rate: 0.0010
Epoch 45/300
1/21  0s 5ms/step - loss: 0.8414 - mae: 0.5908

21/21  0s 1ms/step - loss: 0.7021 - mae: 0.5384 - val_loss: 0.6210 - val_mae: 0.5363 - learning_rate: 0.0010
Epoch 46/300
1/21  0s 5ms/step - loss: 0.4372 - mae: 0.5086
Epoch 46: ReduceLRonPlateau reducing learning rate to 0.0005000000237487257.

21/21  0s 1ms/step - loss: 0.7629 - mae: 0.5499 - val_loss: 0.3737 - val_mae: 0.4225 - learning_rate: 0.0010
Epoch 47/300
1/21  0s 5ms/step - loss: 0.5683 - mae: 0.4610

21/21  0s 1ms/step - loss: 0.6162 - mae: 0.5014 - val_loss: 0.3901 - val_mae: 0.4298 - learning_rate: 5.0000e-04
Epoch 48/300
1/21  0s 5ms/step - loss: 0.7956 - mae: 0.6852

21/21  0s 1ms/step - loss: 0.5506 - mae: 0.5349 - val_loss: 0.2524 - val_mae: 0.3590 - learning_rate: 5.0000e-04
Epoch 49/300
1/21  0s 5ms/step - loss: 0.9879 - mae: 0.7405

21/21  0s 1ms/step - loss: 0.6272 - mae: 0.5501 - val_loss: 0.2697 - val_mae: 0.3754 - learning_rate: 5.0000e-04
Epoch 50/300
1/21  0s 5ms/step - loss: 0.9251 - mae: 0.6503

21/21  0s 1ms/step - loss: 0.6963 - mae: 0.5383 - val_loss: 0.2848 - val_mae: 0.3799 - learning_rate: 5.0000e-04
Epoch 51/300
1/21  0s 5ms/step - loss: 0.3241 - mae: 0.4362

21/21  0s 1ms/step - loss: 0.5633 - mae: 0.5041 - val_loss: 0.3817 - val_mae: 0.4300 - learning_rate: 5.0000e-04
Epoch 52/300
1/21  0s 5ms/step - loss: 0.7545 - mae: 0.6824

21/21  0s 1ms/step - loss: 0.6809 - mae: 0.5492 - val_loss: 0.2388 - val_mae: 0.3687 - learning_rate: 5.0000e-04
Epoch 53/300
1/21  0s 5ms/step - loss: 0.4762 - mae: 0.5196

21/21  0s 1ms/step - loss: 0.5933 - mae: 0.5093 - val_loss: 0.3087 - val_mae: 0.3868 - learning_rate: 5.0000e-04
Epoch 54/300
1/21  0s 5ms/step - loss: 0.3372 - mae: 0.4354

21/21  0s 1ms/step - loss: 0.6179 - mae: 0.5091 - val_loss: 0.2715 - val_mae: 0.3686 - learning_rate: 5.0000e-04
Epoch 55/300
1/21  0s 5ms/step - loss: 0.3879 - mae: 0.4568

21/21  0s 1ms/step - loss: 0.6079 - mae: 0.4841 - val_loss: 0.3747 - val_mae: 0.4228 - learning_rate: 5.0000e-04
Epoch 56/300
1/21  0s 5ms/step - loss: 0.5961 - mae: 0.5553

21/21  0s 1ms/step - loss: 0.5894 - mae: 0.5172 - val_loss: 0.3096 - val_mae: 0.3859 - learning_rate: 5.0000e-04
Epoch 57/300
1/21  0s 5ms/step - loss: 0.3355 - mae: 0.4613

21/21  0s 1ms/step - loss: 0.6994 - mae: 0.5093 - val_loss: 0.3420 - val_mae: 0.4304 - learning_rate: 5.0000e-04
Epoch 58/300
1/21  0s 6ms/step - loss: 0.7497 - mae: 0.5904

21/21  0s 1ms/step - loss: 0.5410 - mae: 0.4976 - val_loss: 0.2603 - val_mae: 0.3572 - learning_rate: 5.0000e-04
Epoch 59/300
1/21  0s 5ms/step - loss: 0.4844 - mae: 0.5532

21/21  0s 1ms/step - loss: 0.5199 - mae: 0.4948 - val_loss: 0.2247 - val_mae: 0.3469 - learning_rate: 5.0000e-04
Epoch 60/300
1/21  0s 5ms/step - loss: 0.6377 - mae: 0.5498

21/21  0s 1ms/step - loss: 0.5778 - mae: 0.4993 - val_loss: 0.4753 - val_mae: 0.4755 - learning_rate: 5.0000e-04
Epoch 61/300
1/21  0s 5ms/step - loss: 0.7603 - mae: 0.6085

21/21  0s 1ms/step - loss: 0.6093 - mae: 0.5051 - val_loss: 0.2861 - val_mae: 0.3873 - learning_rate: 5.0000e-04
Epoch 62/300
1/21  0s 5ms/step - loss: 0.5116 - mae: 0.5255

21/21  0s 1ms/step - loss: 0.5413 - mae: 0.5062 - val_loss: 0.2047 - val_mae: 0.3338 - learning_rate: 5.0000e-04
Epoch 63/300
1/21  0s 5ms/step - loss: 0.3890 - mae: 0.4732

21/21  0s 1ms/step - loss: 0.5301 - mae: 0.4917 - val_loss: 0.2653 - val_mae: 0.3803 - learning_rate: 5.0000e-04
Epoch 64/300
1/21  0s 5ms/step - loss: 0.3133 - mae: 0.4387

21/21  0s 1ms/step - loss: 0.4278 - mae: 0.4609 - val_loss: 0.2461 - val_mae: 0.3618 - learning_rate: 5.0000e-04
Epoch 65/300
1/21  0s 5ms/step - loss: 0.2825 - mae: 0.4306

21/21  0s 1ms/step - loss: 0.4492 - mae: 0.4457 - val_loss: 0.2608 - val_mae: 0.3659 - learning_rate: 5.0000e-04
Epoch 66/300
1/21  0s 5ms/step - loss: 0.3574 - mae: 0.4629

21/21  0s 1ms/step - loss: 0.4712 - mae: 0.4792 - val_loss: 0.2138 - val_mae: 0.3372 - learning_rate: 5.0000e-04
Epoch 67/300
1/21  0s 5ms/step - loss: 0.4448 - mae: 0.5114

21/21  0s 1ms/step - loss: 0.4459 - mae: 0.4609 - val_loss: 0.2171 - val_mae: 0.3393 - learning_rate: 5.0000e-04
Epoch 68/300
1/21  0s 5ms/step - loss: 0.3806 - mae: 0.4360

21/21  0s 1ms/step - loss: 0.5207 - mae: 0.4850 - val_loss: 0.3096 - val_mae: 0.4185 - learning_rate: 5.0000e-04
Epoch 69/300
1/21  0s 5ms/step - loss: 0.4160 - mae: 0.4622

21/21  0s 1ms/step - loss: 0.6494 - mae: 0.5048 - val_loss: 0.3567 - val_mae: 0.4207 - learning_rate: 5.0000e-04
Epoch 70/300
1/21  0s 5ms/step - loss: 0.3869 - mae: 0.4556

21/21  0s 1ms/step - loss: 0.5185 - mae: 0.4819 - val_loss: 0.1917 - val_mae: 0.3341 - learning_rate: 5.0000e-04
Epoch 71/300
1/21  0s 5ms/step - loss: 0.5934 - mae: 0.6044

21/21  0s 1ms/step - loss: 0.4949 - mae: 0.4956 - val_loss: 0.1956 - val_mae: 0.3252 - learning_rate: 5.0000e-04
Epoch 72/300
1/21  0s 5ms/step - loss: 0.4831 - mae: 0.5032

21/21  0s 1ms/step - loss: 0.4960 - mae: 0.4531 - val_loss: 0.3276 - val_mae: 0.4107 - learning_rate: 5.0000e-04
Epoch 73/300
1/21  0s 5ms/step - loss: 0.5760 - mae: 0.5126

21/21  0s 1ms/step - loss: 0.5365 - mae: 0.4865 - val_loss: 0.2483 - val_mae: 0.3600 - learning_rate: 5.0000e-04
Epoch 74/300
1/21  0s 5ms/step - loss: 0.3498 - mae: 0.4601

21/21  0s 1ms/step - loss: 0.4192 - mae: 0.4455 - val_loss: 0.2674 - val_mae: 0.3788 - learning_rate: 5.0000e-04
Epoch 75/300
1/21  0s 5ms/step - loss: 0.5507 - mae: 0.5619

21/21  0s 1ms/step - loss: 0.5644 - mae: 0.4961 - val_loss: 0.4303 - val_mae: 0.4461 - learning_rate: 5.0000e-04
Epoch 76/300
1/21  0s 5ms/step - loss: 0.3173 - mae: 0.3877

21/21  0s 1ms/step - loss: 0.5308 - mae: 0.4683 - val_loss: 0.2119 - val_mae: 0.3262 - learning_rate: 5.0000e-04
Epoch 77/300
1/21  0s 5ms/step - loss: 0.4185 - mae: 0.4463

21/21  0s 1ms/step - loss: 0.4677 - mae: 0.4429 - val_loss: 0.2179 - val_mae: 0.3371 - learning_rate: 5.0000e-04
Epoch 78/300
1/21  0s 5ms/step - loss: 0.6883 - mae: 0.5810

21/21  0s 1ms/step - loss: 0.4590 - mae: 0.4593 - val_loss: 0.1890 - val_mae: 0.3072 - learning_rate: 5.0000e-04
Epoch 79/300
1/21  0s 5ms/step - loss: 0.3578 - mae: 0.4390

21/21  0s 1ms/step - loss: 0.4301 - mae: 0.4398 - val_loss: 0.3316 - val_mae: 0.4061 - learning_rate: 5.0000e-04
Epoch 80/300
1/21  0s 5ms/step - loss: 0.5011 - mae: 0.4974

21/21  0s 1ms/step - loss: 0.7376 - mae: 0.5242 - val_loss: 0.2744 - val_mae: 0.3928 - learning_rate: 5.0000e-04
Epoch 81/300
1/21  0s 5ms/step - loss: 0.4254 - mae: 0.4873

21/21  0s 1ms/step - loss: 0.4089 - mae: 0.4347 - val_loss: 0.2017 - val_mae: 0.3192 - learning_rate: 5.0000e-04
Epoch 82/300
1/21  0s 5ms/step - loss: 0.3074 - mae: 0.4136

21/21  0s 1ms/step - loss: 0.3743 - mae: 0.4371 - val_loss: 0.2613 - val_mae: 0.3785 - learning_rate: 5.0000e-04
Epoch 83/300
1/21  0s 5ms/step - loss: 0.5245 - mae: 0.5820

21/21  0s 1ms/step - loss: 0.4847 - mae: 0.4800 - val_loss: 0.1845 - val_mae: 0.3209 - learning_rate: 5.0000e-04
Epoch 84/300
1/21  0s 5ms/step - loss: 0.3344 - mae: 0.4409

21/21  0s 1ms/step - loss: 0.3696 - mae: 0.4225 - val_loss: 0.1671 - val_mae: 0.2988 - learning_rate: 5.0000e-04
Epoch 85/300
1/21  0s 5ms/step - loss: 0.2261 - mae: 0.3543

21/21  0s 1ms/step - loss: 0.4272 - mae: 0.4323 - val_loss: 0.1947 - val_mae: 0.3320 - learning_rate: 5.0000e-04
Epoch 86/300
1/21  0s 5ms/step - loss: 0.1886 - mae: 0.3483

21/21  0s 1ms/step - loss: 0.3775 - mae: 0.4152 - val_loss: 0.2424 - val_mae: 0.3386 - learning_rate: 5.0000e-04
Epoch 87/300
1/21  0s 5ms/step - loss: 0.2052 - mae: 0.3725

21/21  0s 1ms/step - loss: 0.3420 - mae: 0.4170 - val_loss: 0.1919 - val_mae: 0.3230 - learning_rate: 5.0000e-04
Epoch 88/300
1/21  0s 5ms/step - loss: 0.1963 - mae: 0.3596

21/21  0s 1ms/step - loss: 0.3755 - mae: 0.4218 - val_loss: 0.2457 - val_mae: 0.3561 - learning_rate: 5.0000e-04
Epoch 89/300
1/21  0s 5ms/step - loss: 0.4042 - mae: 0.4877

21/21  0s 1ms/step - loss: 0.4246 - mae: 0.4173 - val_loss: 0.3167 - val_mae: 0.4050 - learning_rate: 5.0000e-04
Epoch 90/300
1/21  0s 5ms/step - loss: 0.2432 - mae: 0.3666

21/21  0s 1ms/step - loss: 0.4607 - mae: 0.4416 - val_loss: 0.2352 - val_mae: 0.3624 - learning_rate: 5.0000e-04
Epoch 91/300
1/21  0s 5ms/step - loss: 0.6991 - mae: 0.5397

21/21  0s 1ms/step - loss: 0.4913 - mae: 0.4461 - val_loss: 0.1706 - val_mae: 0.3053 - learning_rate: 5.0000e-04
Epoch 92/300
1/21  0s 5ms/step - loss: 0.3649 - mae: 0.4519

21/21  0s 1ms/step - loss: 0.3793 - mae: 0.4283 - val_loss: 0.1643 - val_mae: 0.2988 - learning_rate: 5.0000e-04
Epoch 93/300
1/21  0s 5ms/step - loss: 0.1849 - mae: 0.3179

21/21  0s 1ms/step - loss: 0.3844 - mae: 0.4239 - val_loss: 0.1692 - val_mae: 0.3125 - learning_rate: 5.0000e-04
Epoch 94/300
1/21  0s 5ms/step - loss: 0.3919 - mae: 0.4494

21/21  0s 4ms/step - loss: 0.4122 - mae: 0.4313 - val_loss: 0.1704 - val_mae: 0.3094 - learning_rate: 5.0000e-04
Epoch 95/300
1/21  0s 8ms/step - loss: 0.2800 - mae: 0.4096

21/21  0s 2ms/step - loss: 0.3643 - mae: 0.4265 - val_loss: 0.2481 - val_mae: 0.3625 - learning_rate: 5.0000e-04
Epoch 96/300
1/21  0s 6ms/step - loss: 0.2306 - mae: 0.3780

21/21  0s 1ms/step - loss: 0.4095 - mae: 0.4291 - val_loss: 0.3300 - val_mae: 0.3861 - learning_rate: 5.0000e-04
Epoch 97/300
1/21  0s 5ms/step - loss: 0.3997 - mae: 0.4496

21/21  0s 1ms/step - loss: 0.4729 - mae: 0.4380 - val_loss: 0.3721 - val_mae: 0.4379 - learning_rate: 5.0000e-04
Epoch 98/300
1/21  0s 5ms/step - loss: 0.9288 - mae: 0.6125

21/21  0s 1ms/step - loss: 0.6194 - mae: 0.4806 - val_loss: 0.2279 - val_mae: 0.3563 - learning_rate: 5.0000e-04
Epoch 99/300
1/21  0s 5ms/step - loss: 0.3092 - mae: 0.4449

21/21  0s 1ms/step - loss: 0.4500 - mae: 0.4439 - val_loss: 0.1485 - val_mae: 0.2944 - learning_rate: 5.0000e-04
Epoch 100/300
1/21  0s 5ms/step - loss: 0.3039 - mae: 0.3935

21/21  0s 1ms/step - loss: 0.3779 - mae: 0.4009 - val_loss: 0.2444 - val_mae: 0.3471 - learning_rate: 5.0000e-04
Epoch 101/300
1/21  0s 5ms/step - loss: 0.4100 - mae: 0.4411

21/21  0s 1ms/step - loss: 0.4481 - mae: 0.4493 - val_loss: 0.2429 - val_mae: 0.3672 - learning_rate: 5.0000e-04
Epoch 102/300
1/21  0s 5ms/step - loss: 0.2395 - mae: 0.3798

21/21  0s 2ms/step - loss: 0.3700 - mae: 0.4279 - val_loss: 0.2168 - val_mae: 0.3318 - learning_rate: 5.0000e-04
Epoch 103/300
1/21  0s 15ms/step - loss: 0.3643 - mae: 0.4664

21/21  0s 2ms/step - loss: 0.3843 - mae: 0.4311 - val_loss: 0.2202 - val_mae: 0.3428 - learning_rate: 5.0000e-04
Epoch 104/300
1/21  0s 6ms/step - loss: 0.6392 - mae: 0.5409

21/21  0s 1ms/step - loss: 0.4178 - mae: 0.4361 - val_loss: 0.2277 - val_mae: 0.3571 - learning_rate: 5.0000e-04
Epoch 105/300
1/21  0s 6ms/step - loss: 0.6702 - mae: 0.5669

21/21  0s 1ms/step - loss: 0.4132 - mae: 0.4315 - val_loss: 0.3140 - val_mae: 0.4014 - learning_rate: 5.0000e-04
Epoch 106/300
1/21  0s 11ms/step - loss: 0.5094 - mae: 0.4617

21/21  0s 1ms/step - loss: 0.4276 - mae: 0.4344 - val_loss: 0.4073 - val_mae: 0.4396 - learning_rate: 5.0000e-04
Epoch 107/300
1/21  0s 6ms/step - loss: 0.2698 - mae: 0.3948

21/21  0s 1ms/step - loss: 0.5313 - mae: 0.4319 - val_loss: 0.1971 - val_mae: 0.3211 - learning_rate: 5.0000e-04
Epoch 108/300
1/21  0s 7ms/step - loss: 0.2303 - mae: 0.3964

21/21  0s 1ms/step - loss: 0.4115 - mae: 0.4264 - val_loss: 0.1659 - val_mae: 0.3049 - learning_rate: 5.0000e-04
Epoch 109/300
1/21  0s 6ms/step - loss: 0.4814 - mae: 0.4409
Epoch 109: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

21/21  0s 1ms/step - loss: 0.4747 - mae: 0.4420 - val_loss: 0.2053 - val_mae: 0.3443 - learning_rate: 5.0000e-04
Epoch 110/300

1/21 ————— 0s 6ms/step - loss: 0.2391 - mae: 0.3512

21/21 ————— 0s 1ms/step - loss: 0.4107 - mae: 0.4206 - val_loss: 0.1524 - val_mae: 0.2930 - learning_rate: 2.5000e-04
Epoch 111/300

1/21 ————— 0s 6ms/step - loss: 0.2282 - mae: 0.3749

21/21 ————— 0s 1ms/step - loss: 0.3632 - mae: 0.4099 - val_loss: 0.2140 - val_mae: 0.3392 - learning_rate: 2.5000e-04
Epoch 112/300

1/21 ————— 0s 7ms/step - loss: 0.2854 - mae: 0.4020

21/21 ————— 0s 1ms/step - loss: 0.4149 - mae: 0.4193 - val_loss: 0.1945 - val_mae: 0.3218 - learning_rate: 2.5000e-04
Epoch 113/300

1/21 ————— 0s 7ms/step - loss: 0.1887 - mae: 0.3137

21/21 ————— 0s 1ms/step - loss: 0.3128 - mae: 0.3838 - val_loss: 0.1662 - val_mae: 0.3005 - learning_rate: 2.5000e-04
Epoch 114/300

1/21 ————— 0s 6ms/step - loss: 0.1880 - mae: 0.3341

21/21 ————— 0s 1ms/step - loss: 0.3346 - mae: 0.3905 - val_loss: 0.2123 - val_mae: 0.3432 - learning_rate: 2.5000e-04
Epoch 115/300

1/21 ————— 0s 6ms/step - loss: 0.2642 - mae: 0.3774

21/21 ————— 0s 1ms/step - loss: 0.3295 - mae: 0.3864 - val_loss: 0.1635 - val_mae: 0.2947 - learning_rate: 2.5000e-04
Epoch 116/300

1/21 ————— 0s 6ms/step - loss: 0.3359 - mae: 0.4547

21/21 ————— 0s 1ms/step - loss: 0.3234 - mae: 0.3997 - val_loss: 0.2598 - val_mae: 0.3670 - learning_rate: 2.5000e-04
Epoch 117/300

1/21 ————— 0s 6ms/step - loss: 0.2370 - mae: 0.3711

21/21 ————— 0s 1ms/step - loss: 0.3580 - mae: 0.4045 - val_loss: 0.1659 - val_mae: 0.2940 - learning_rate: 2.5000e-04
Epoch 118/300

1/21 ————— 0s 5ms/step - loss: 0.2397 - mae: 0.4008

21/21 ————— 0s 1ms/step - loss: 0.3906 - mae: 0.4162 - val_loss: 0.2899 - val_mae: 0.3886 - learning_rate: 2.5000e-04
Epoch 119/300

1/21 ————— 0s 5ms/step - loss: 0.2544 - mae: 0.3474
Epoch 119: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

21/21 ————— 0s 1ms/step - loss: 0.3728 - mae: 0.4007 - val_loss: 0.1768 - val_mae: 0.3042 - learning_rate: 2.5000e-04
Epoch 120/300

1/21 ————— 0s 5ms/step - loss: 0.2863 - mae: 0.4056

21/21 ————— 0s 1ms/step - loss: 0.3900 - mae: 0.4083 - val_loss: 0.2562 - val_mae: 0.3549 - learning_rate: 1.2500e-04

Epoch 121/300
 1/21 ————— 0s 6ms/step - loss: 0.2544 - mae: 0.4083

21/21 ————— 0s 1ms/step - loss: 0.3654 - mae: 0.4225 - val_loss: 0.2316 - val_mae: 0.3427 - learning_rate: 1.2500e-04

Epoch 122/300
 1/21 ————— 0s 5ms/step - loss: 0.3415 - mae: 0.4357

21/21 ————— 0s 1ms/step - loss: 0.4076 - mae: 0.4044 - val_loss: 0.2025 - val_mae: 0.3241 - learning_rate: 1.2500e-04

Epoch 123/300
 1/21 ————— 0s 5ms/step - loss: 0.2242 - mae: 0.3724

21/21 ————— 0s 1ms/step - loss: 0.2811 - mae: 0.3735 - val_loss: 0.2079 - val_mae: 0.3291 - learning_rate: 1.2500e-04

Epoch 124/300
 1/21 ————— 0s 5ms/step - loss: 0.3025 - mae: 0.4164

21/21 ————— 0s 1ms/step - loss: 0.3858 - mae: 0.4150 - val_loss: 0.1845 - val_mae: 0.3102 - learning_rate: 1.2500e-04

Epoch 125/300
 1/21 ————— 0s 5ms/step - loss: 0.2832 - mae: 0.3715

21/21 ————— 0s 1ms/step - loss: 0.3123 - mae: 0.3836 - val_loss: 0.2298 - val_mae: 0.3511 - learning_rate: 1.2500e-04

Epoch 126/300
 1/21 ————— 0s 5ms/step - loss: 0.3244 - mae: 0.4198

21/21 ————— 0s 1ms/step - loss: 0.3393 - mae: 0.3939 - val_loss: 0.1652 - val_mae: 0.2969 - learning_rate: 1.2500e-04

Epoch 127/300
 1/21 ————— 0s 5ms/step - loss: 0.4425 - mae: 0.5118

21/21 ————— 0s 1ms/step - loss: 0.4188 - mae: 0.4437 - val_loss: 0.1914 - val_mae: 0.3188 - learning_rate: 1.2500e-04

Epoch 128/300
 1/21 ————— 0s 5ms/step - loss: 0.2784 - mae: 0.4443

21/21 ————— 0s 1ms/step - loss: 0.3483 - mae: 0.4048 - val_loss: 0.1999 - val_mae: 0.3308 - learning_rate: 1.2500e-04

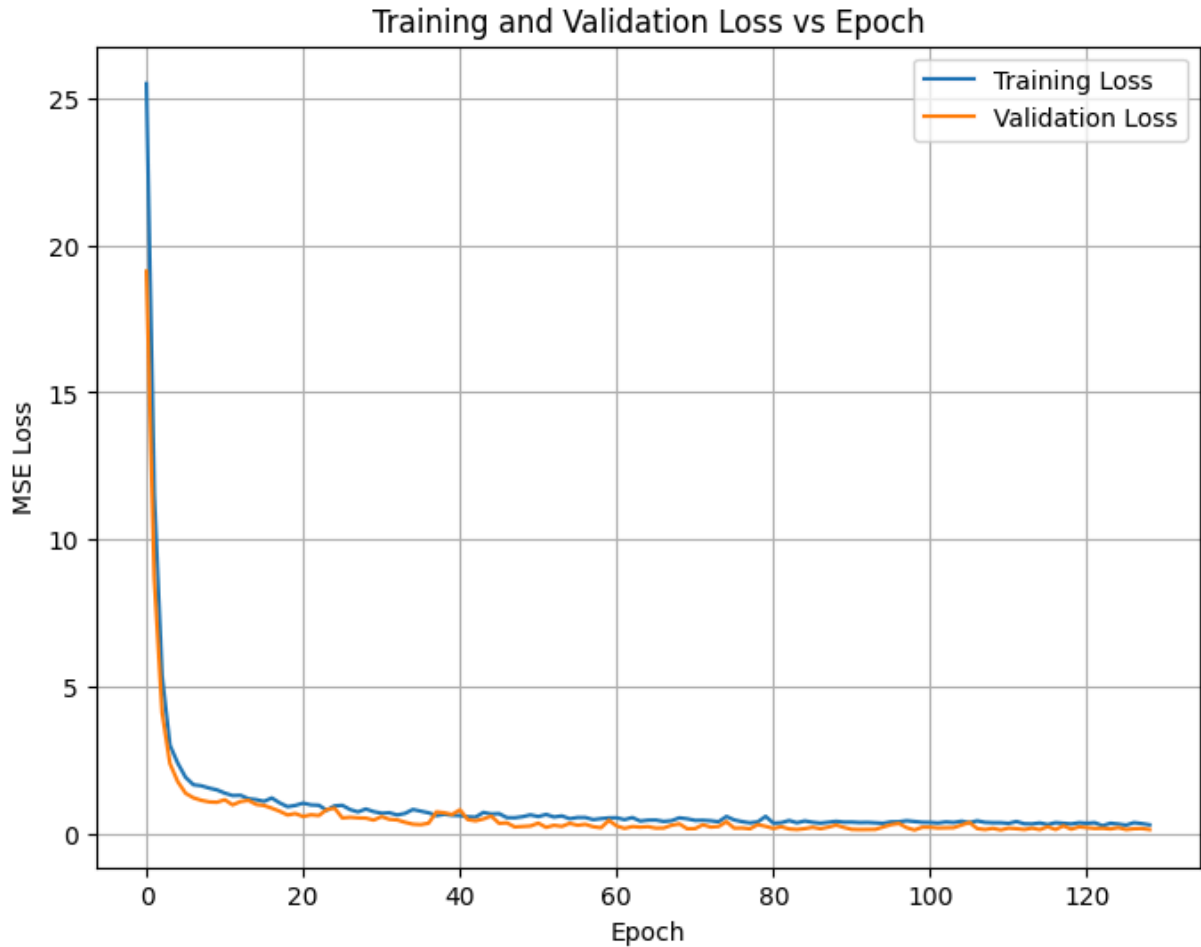
Epoch 129/300
 1/21 ————— 0s 5ms/step - loss: 0.2582 - mae: 0.3575

Epoch 129: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

21/21 ————— 0s 1ms/step - loss: 0.3203 - mae: 0.3827 - val_loss: 0.1643 - val_mae: 0.2997 - learning_rate: 1.2500e-04

```
In [13]: plt.figure(figsize=(8, 6))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training and Validation Loss vs Epoch")
plt.legend()
plt.grid(True)
```

```
plt.savefig('plots/training_and_validation_loss_vs_epoch.png', dpi=150, bbox
plt.show()
```



```
In [14]: y_pred = baseline_model.predict(X_test_scaled).flatten()
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE : {mse:.4f}")
print(f"Test RMSE : {rmse:.4f}")
print(f"Test MAE : {mae:.4f}")
print(f"Test R^2 : {r2:.4f}")
```

1/5 — 0s 18ms/step

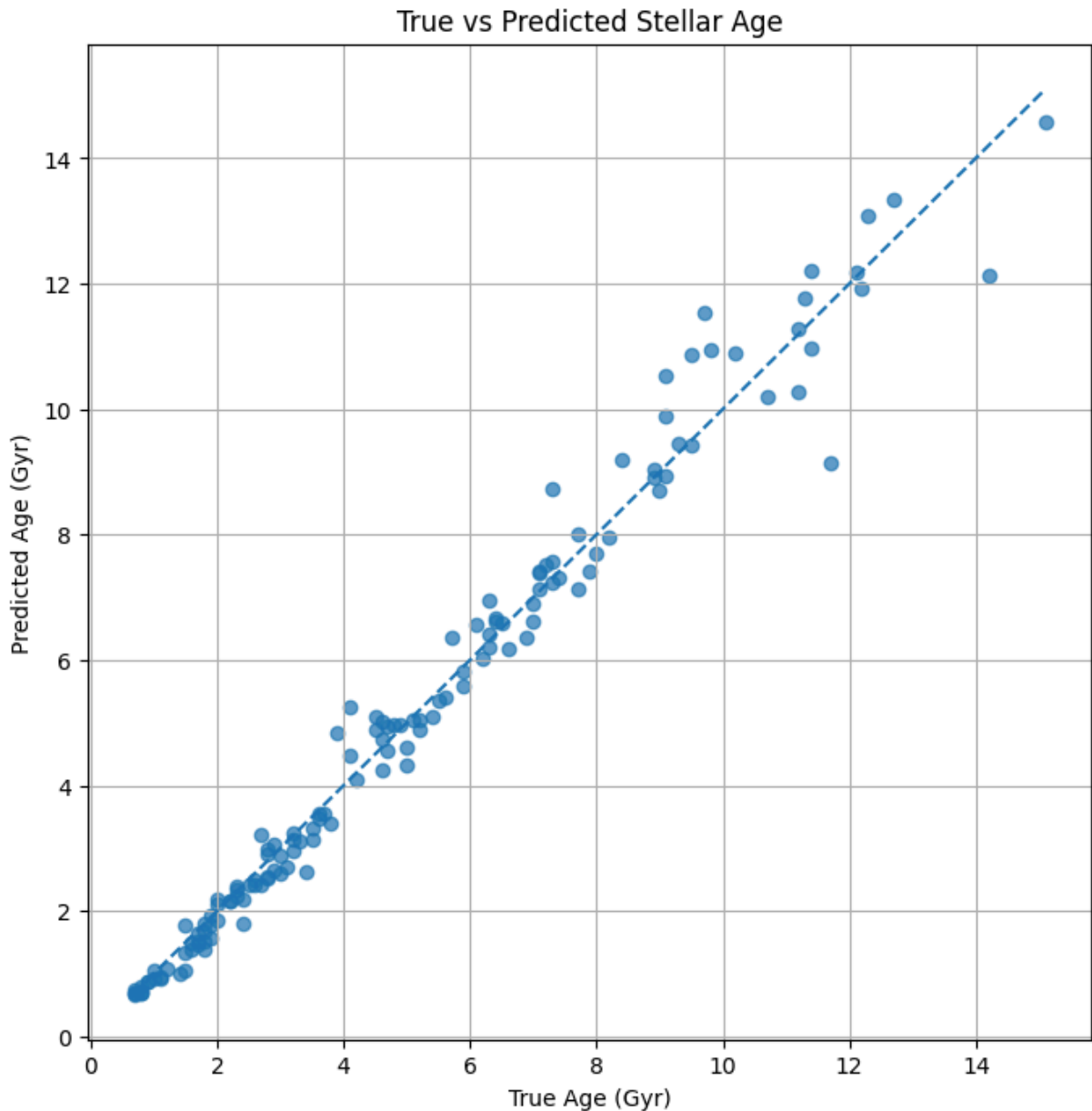
5/5 — 0s 4ms/step

Test MSE : 0.2703
Test RMSE : 0.5199
Test MAE : 0.3380
Test R^2 : 0.9767

```
In [15]: plt.figure(figsize=(8, 8))
plt.scatter(y_test, y_pred, alpha=0.7)
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle="--")
```

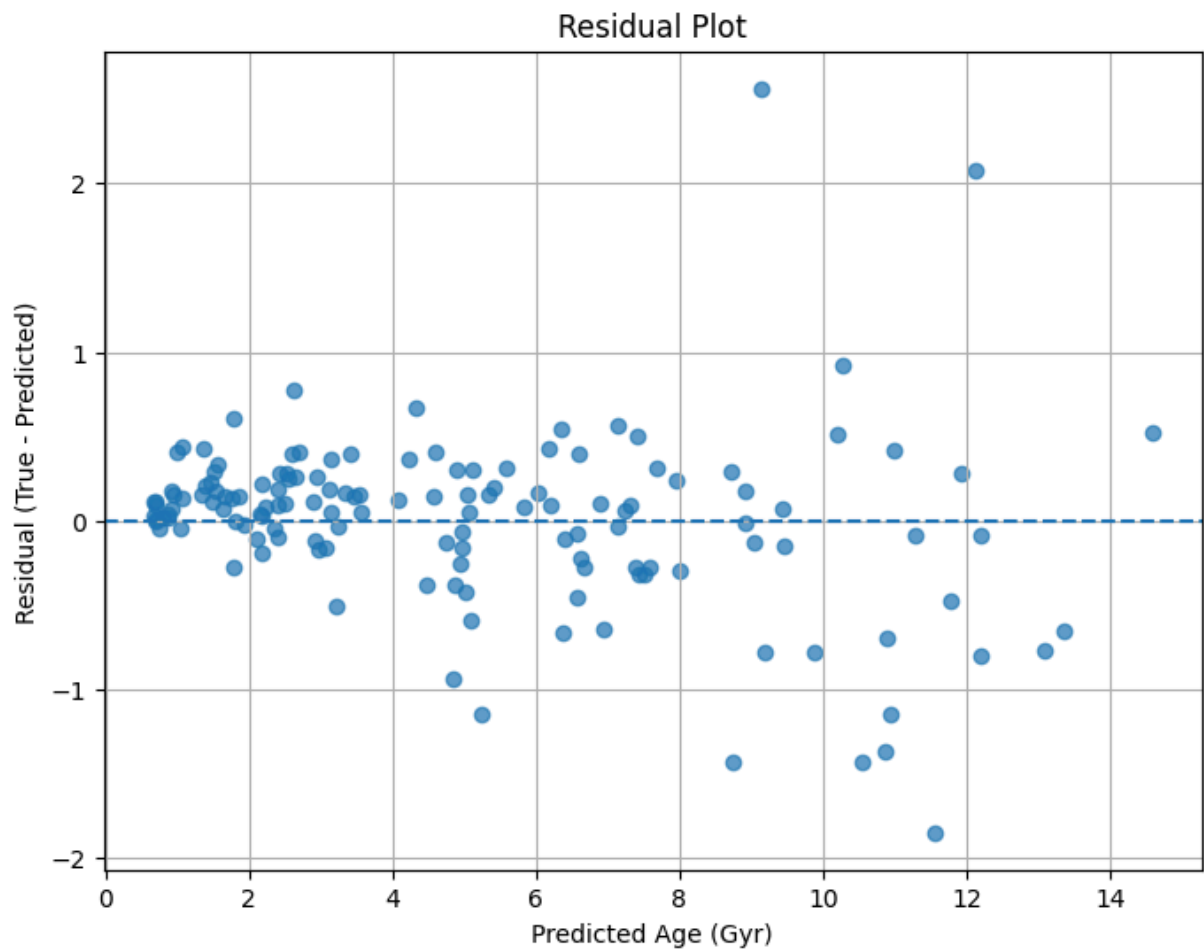


```
plt.xlabel("True Age (Gyr)")
plt.ylabel("Predicted Age (Gyr)")
plt.title("True vs Predicted Stellar Age")
plt.grid(True)
plt.savefig('plots/true_vs_predicted_stellar_age.png', dpi=150, bbox_inches=
plt.show())
```

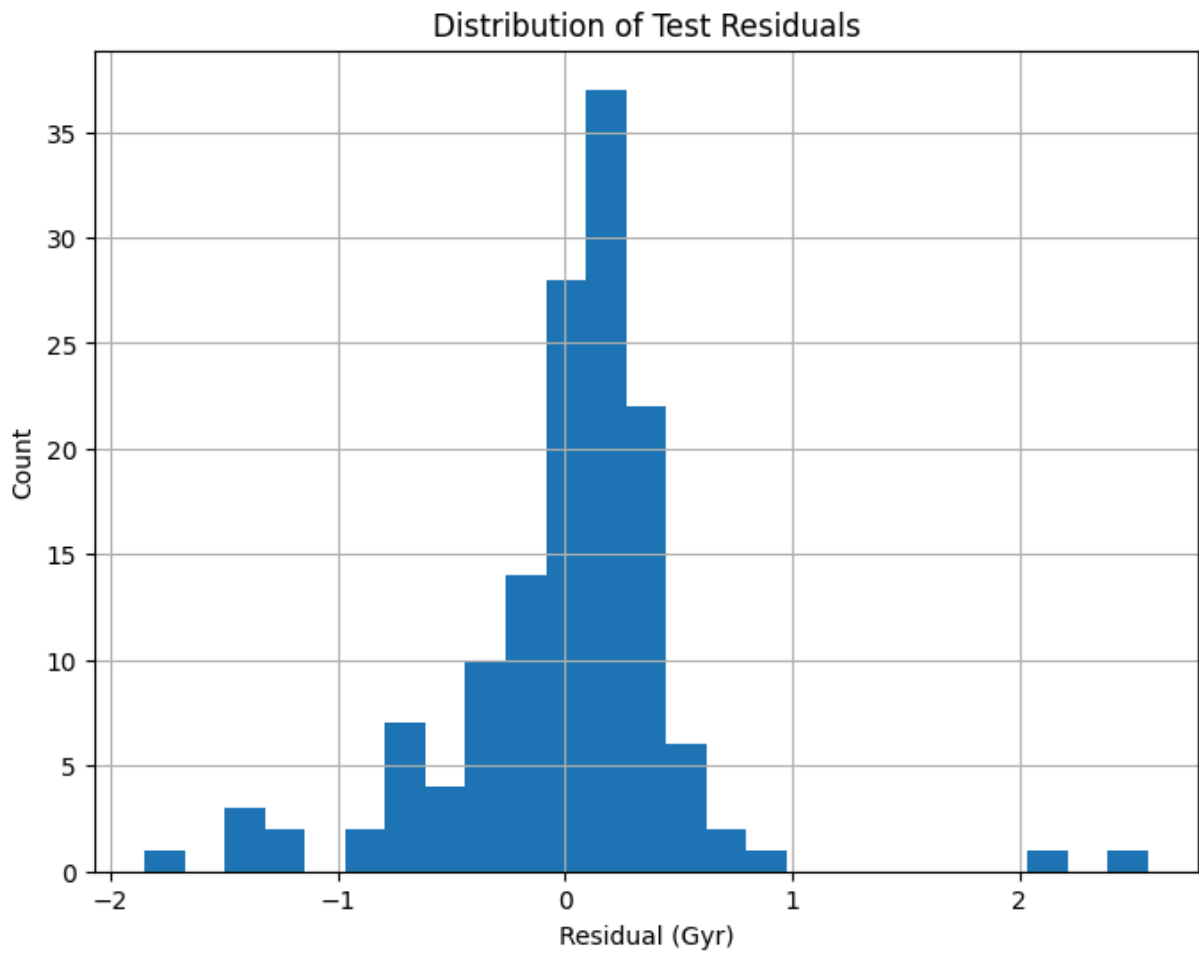


```
In [16]: residuals = y_test - y_pred

plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.7)
plt.axhline(0, linestyle="--")
plt.xlabel("Predicted Age (Gyr)")
plt.ylabel("Residual (True - Predicted)")
plt.title("Residual Plot")
plt.grid(True)
plt.savefig('plots/residual_plot.png', dpi=150, bbox_inches='tight')
plt.show()
```

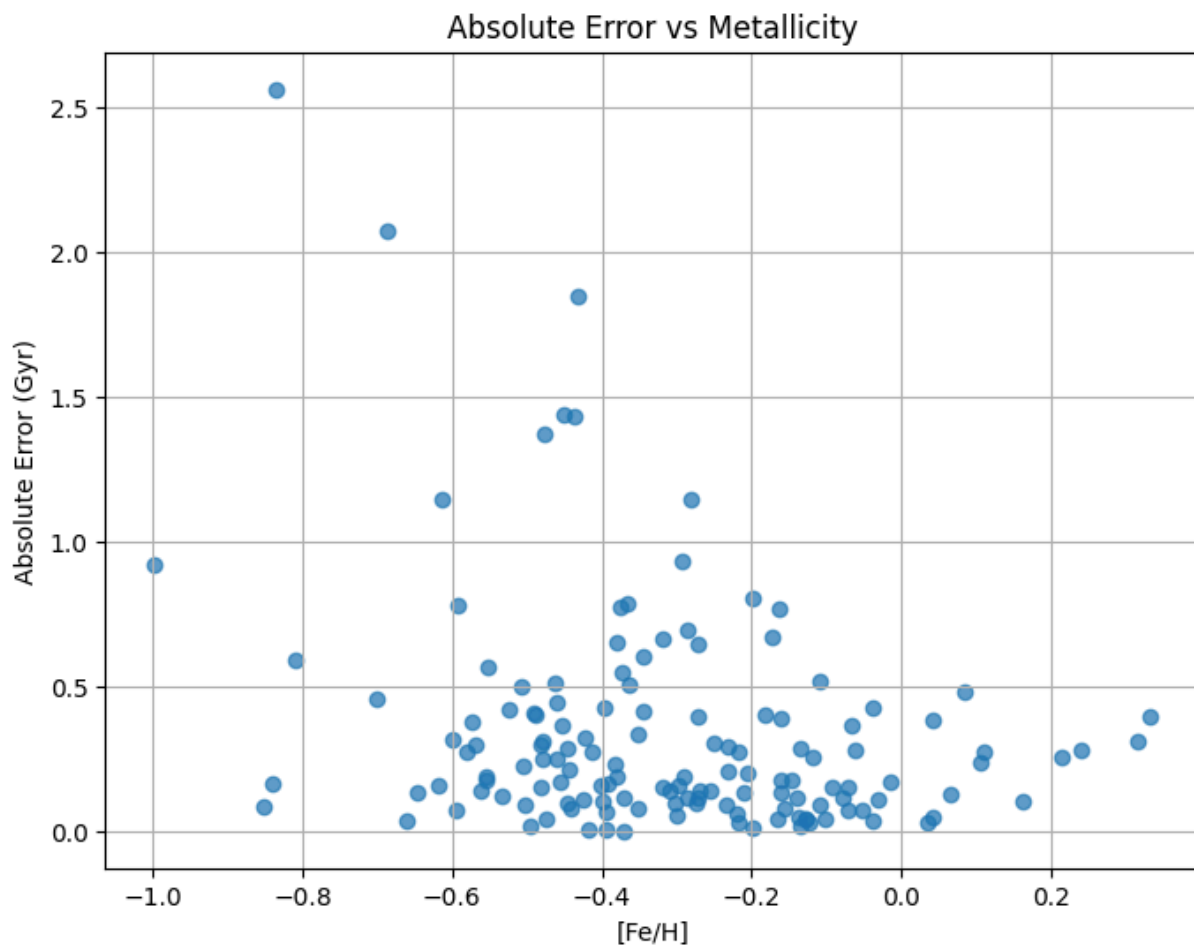


```
In [17]: plt.figure(figsize=(8, 6))
plt.hist(residuals, bins=25)
plt.xlabel("Residual (Gyr)")
plt.ylabel("Count")
plt.title("Distribution of Test Residuals")
plt.grid(True)
plt.savefig('plots/distribution_of_test_residuals.png', dpi=150, bbox_inches='tight')
plt.show()
```



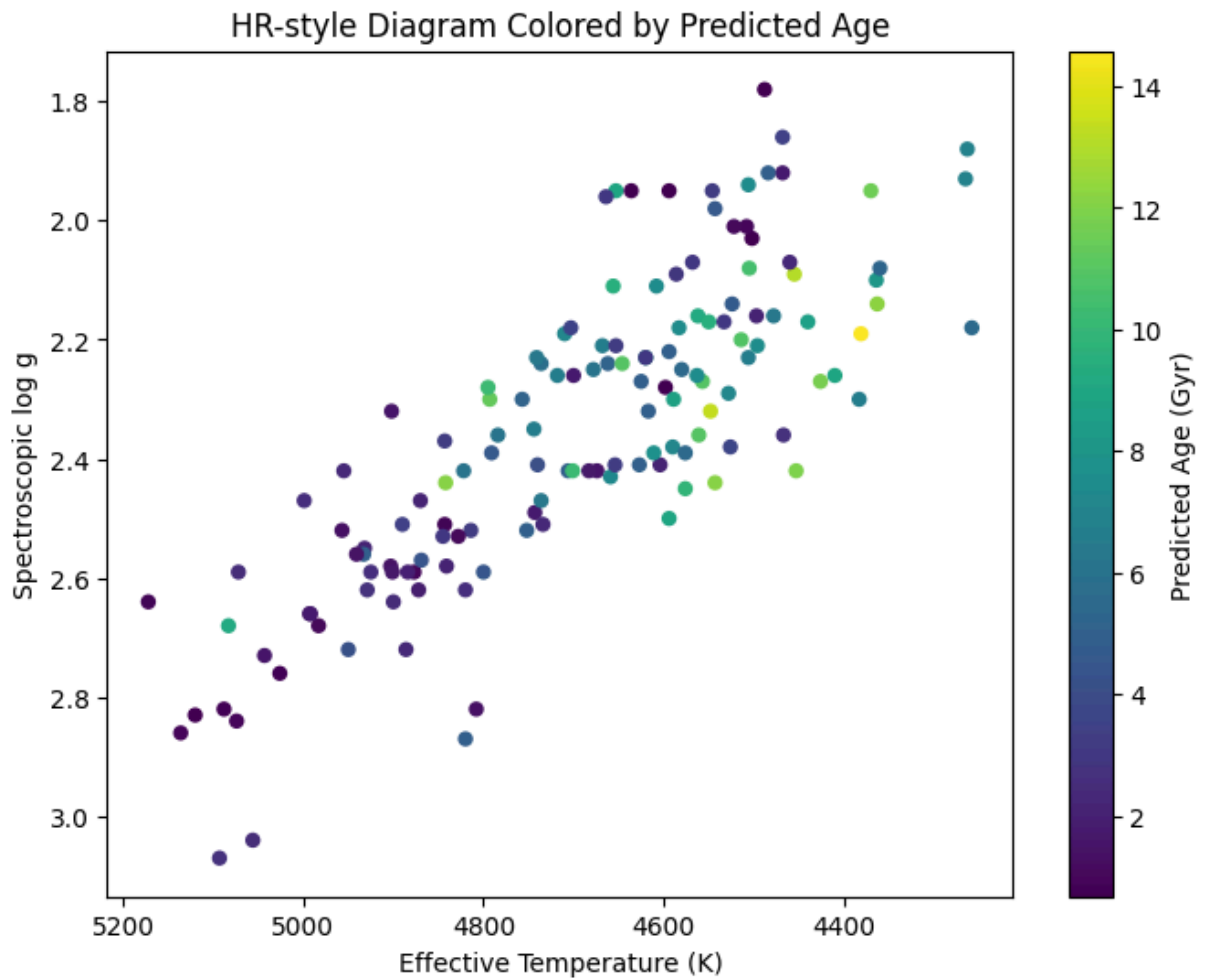
```
In [18]: test_results = X_test.copy()
test_results["true_age"] = y_test.values
test_results["pred_age"] = y_pred
test_results["abs_error"] = np.abs(test_results["true_age"] - test_results["pred_age"])

plt.figure(figsize=(8, 6))
plt.scatter(test_results["feh"], test_results["abs_error"], alpha=0.7)
plt.xlabel("[Fe/H]")
plt.ylabel("Absolute Error (Gyr)")
plt.title("Absolute Error vs Metallicity")
plt.grid(True)
plt.savefig('plots/absolute_error_vs_metallicity.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [19]: test_results["teff"] = X_test["teff"].values
test_results["logg_spec"] = X_test["logg_spec"].values

plt.figure(figsize=(8, 6))
plt.scatter(test_results["teff"], test_results["logg_spec"], c=test_results["age"])
plt.gca().invert_xaxis()
plt.gca().invert_yaxis()
plt.xlabel("Effective Temperature (K)")
plt.ylabel("Spectroscopic log g")
plt.title("HR-style Diagram Colored by Predicted Age")
plt.colorbar(label="Predicted Age (Gyr)")
plt.savefig('plots/hr_style_diagram_colored_by_predicted_age.png', dpi=150,
plt.show()
```



```
In [20]: def build_model(hidden_units=[64, 32], dropout_rate=0.2, learning_rate=1e-3)
    model = Sequential()
    model.add(Input(shape=(X_train_scaled.shape[1],)))

    for units in hidden_units:
        model.add(Dense(units, activation="relu"))
        model.add(Dropout(dropout_rate))

    model.add(Dense(1, activation="linear"))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse",
        metrics=["mae"]
    )
    return model

architectures = {
    "small": [32, 16],
    "medium": [128, 64, 32],
    "large": [256, 128, 64, 32]
}

comparison_results = []
```

```

for name, arch in architectures.items():
    model = build_model(hidden_units=arch, dropout_rate=0.2, learning_rate=1e-5)

    history_tmp = model.fit(
        X_train_scaled, y_train,
        validation_data=(X_val_scaled, y_val),
        epochs=200,
        batch_size=32,
        callbacks=[early_stop, reduce_lr],
        verbose=0
    )

    pred_tmp = model.predict(X_test_scaled, verbose=0).flatten()
    rmse_tmp = np.sqrt(mean_squared_error(y_test, pred_tmp))
    mae_tmp = mean_absolute_error(y_test, pred_tmp)
    r2_tmp = r2_score(y_test, pred_tmp)

    comparison_results.append({
        "model": name,
        "architecture": arch,
        "rmse": rmse_tmp,
        "mae": mae_tmp,
        "r2": r2_tmp
    })

comparison_df = pd.DataFrame(comparison_results)
print(comparison_df)

```

Epoch 122: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 143: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 153: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 163: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 49: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 74: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 85: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 95: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 107: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 117: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

Epoch 127: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x14b2b08b0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

WARNING:tensorflow:5 out of the last 11 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x14b2b08b0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

Epoch 38: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 63: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 84: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

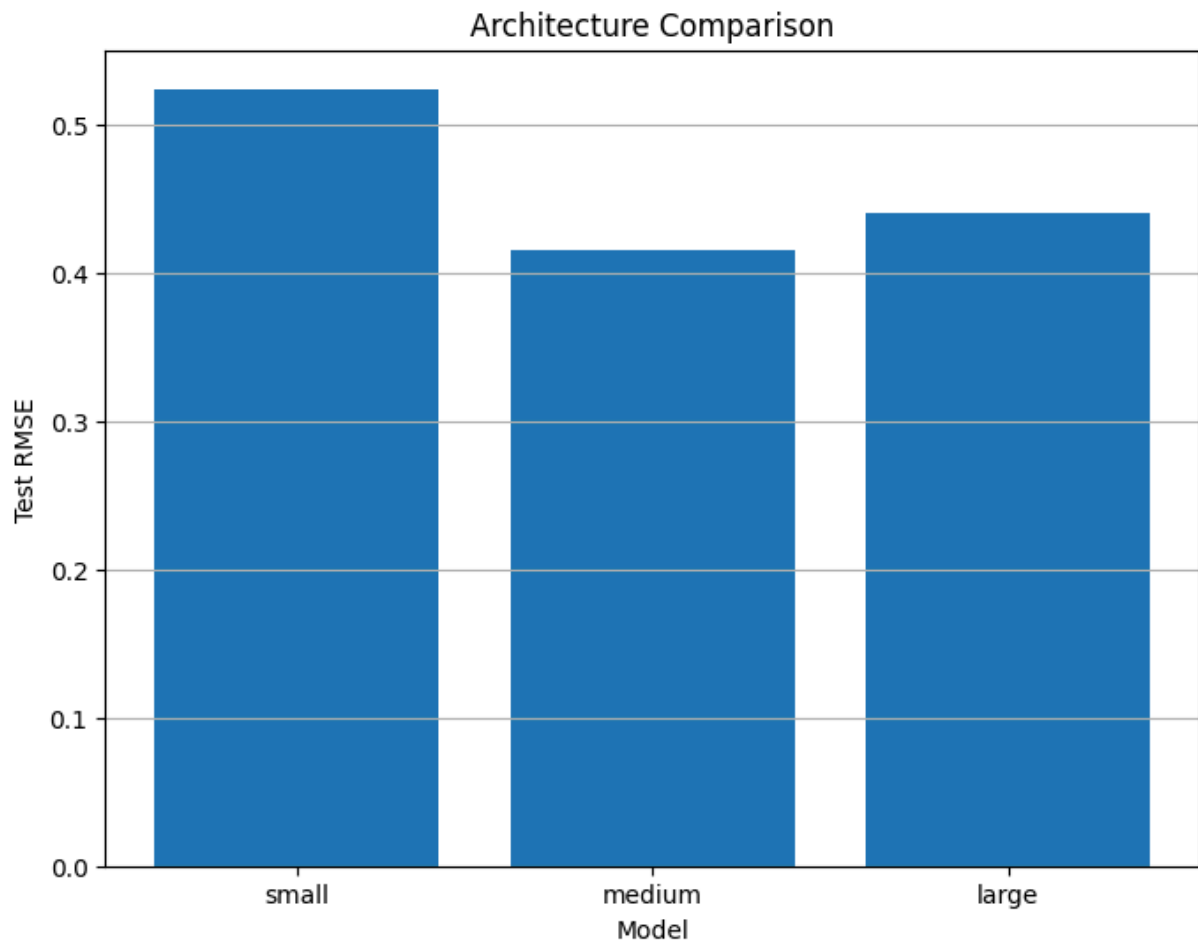
Epoch 114: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 124: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 134: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

	model	architecture	rmse	mae	r2
0	small	[32, 16]	0.524480	0.336688	0.976315
1	medium	[128, 64, 32]	0.415911	0.285510	0.985106
2	large	[256, 128, 64, 32]	0.440479	0.294700	0.983294

```
In [21]: plt.figure(figsize=(8, 6))
plt.bar(comparison_df["model"], comparison_df["rmse"])
plt.xlabel("Model")
plt.ylabel("Test RMSE")
plt.title("Architecture Comparison")
plt.grid(axis="y")
plt.savefig('plots/architecture_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

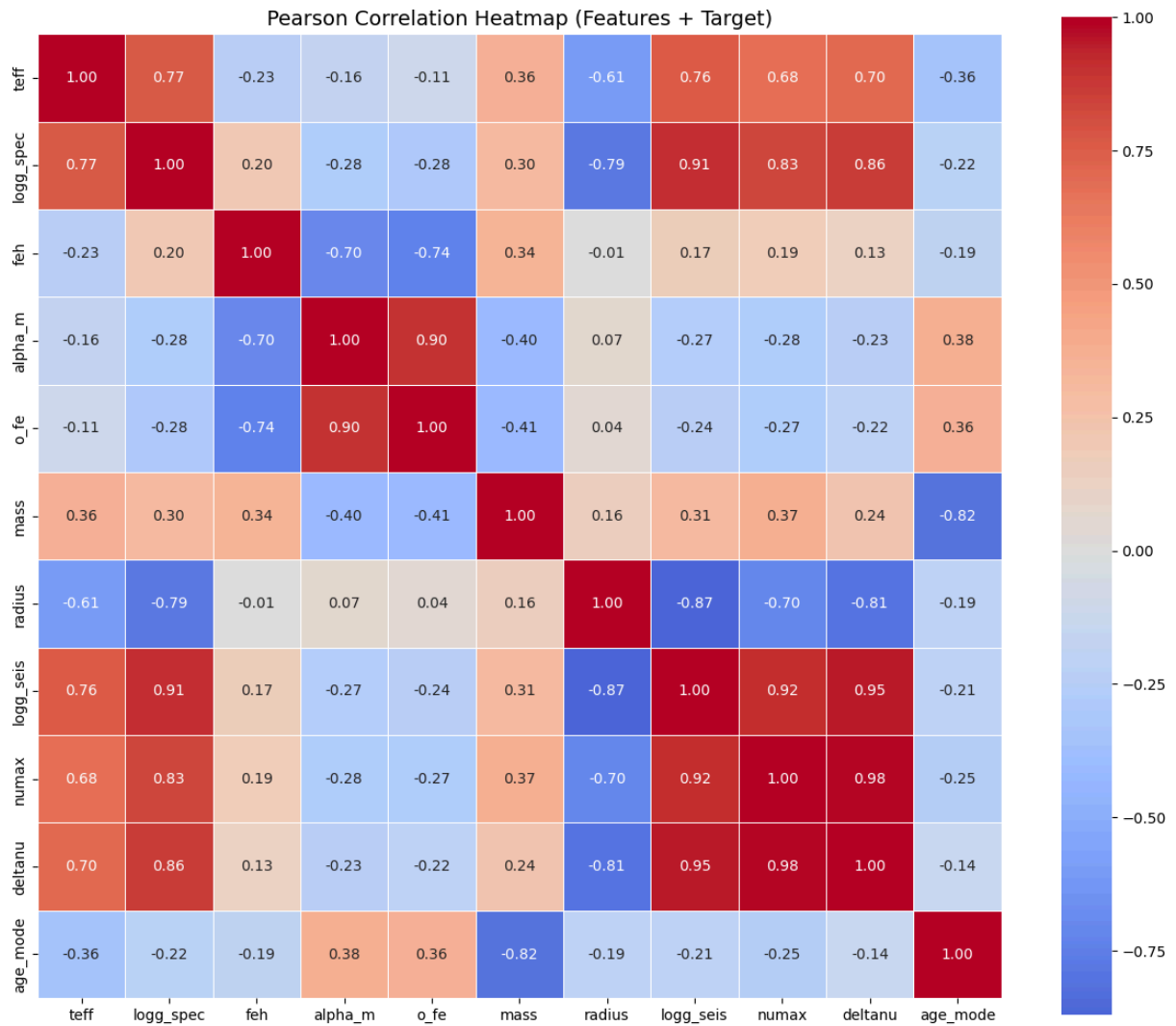


Additional Diagnostic Visualisations

```
In [22]: # 1. Correlation Heatmap of Input Features + Target
import seaborn as sns

corr_cols = feature_cols + [target_col]
corr_matrix = data[corr_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.5
)
plt.title('Pearson Correlation Heatmap (Features + Target)', fontsize=14)
plt.tight_layout()
plt.savefig('plots/pearson_correlation_heatmap_features_target.png', dpi=150)
plt.show()
```

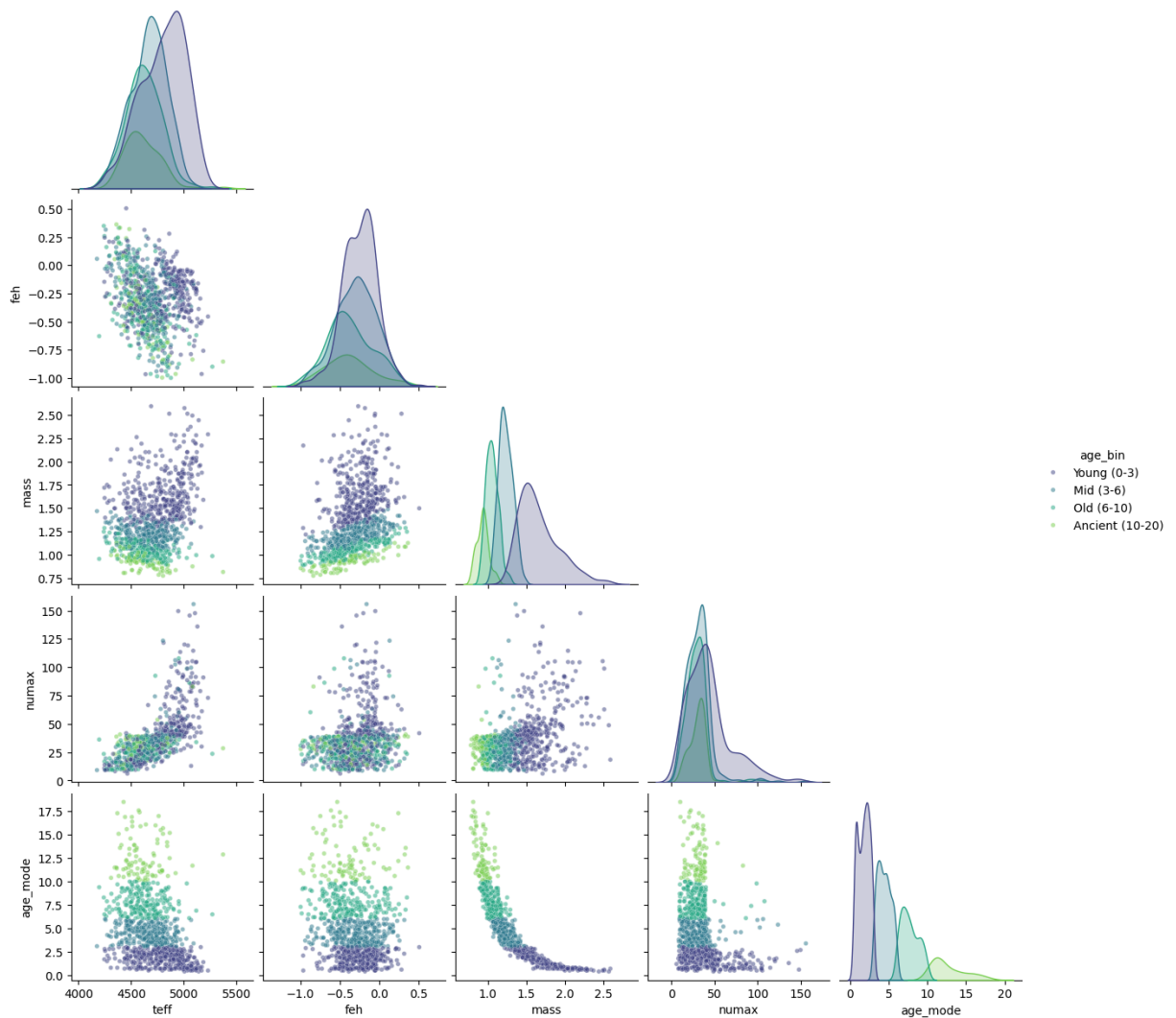



```
In [23]: # 2. Pair Plot of Selected Key Features coloured by Age
pair_cols = ['teff', 'feh', 'mass', 'numax', 'age_mode']
pair_data = data[pair_cols].copy()

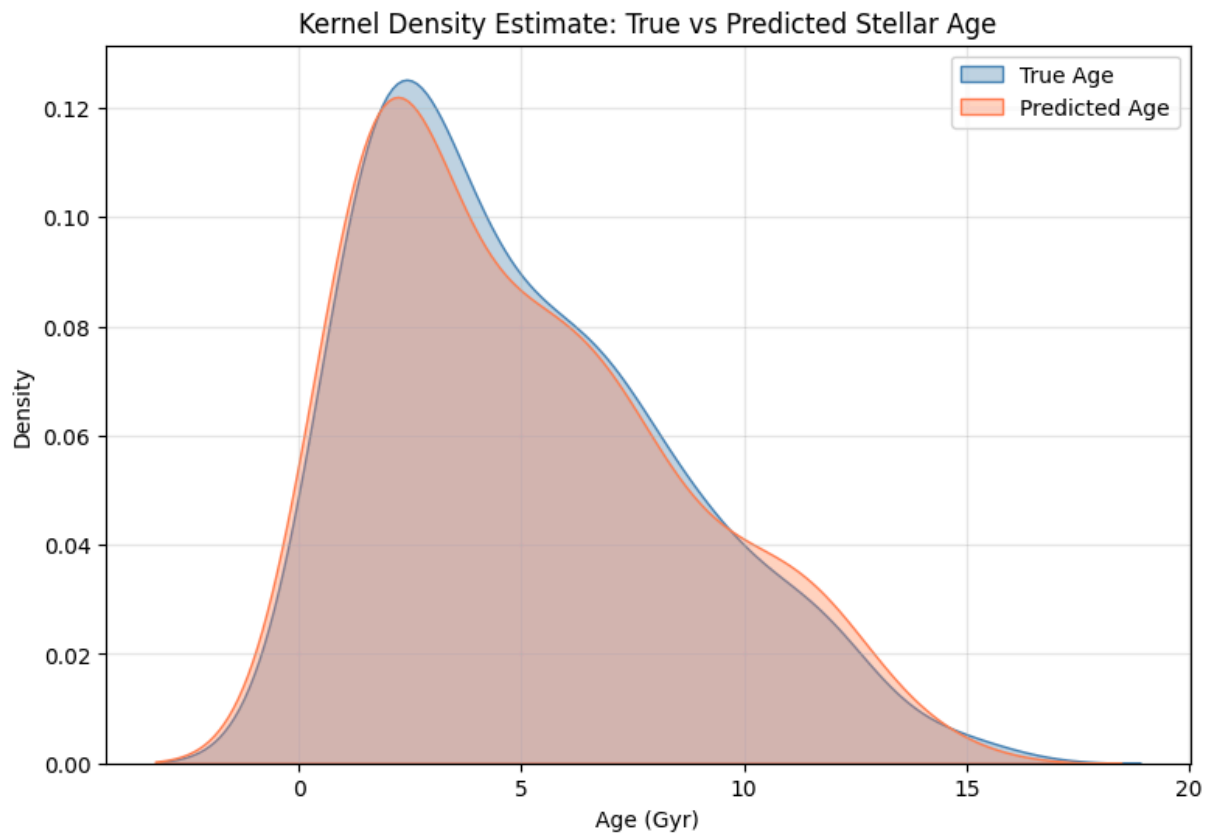
# Bin age into categories for colouring
pair_data['age_bin'] = pd.cut(
    pair_data['age_mode'],
    bins=[0, 3, 6, 10, 20],
    labels=['Young (0-3)', 'Mid (3-6)', 'Old (6-10)', 'Ancient (10-20)']
)

g = sns.pairplot(
    pair_data,
    hue='age_bin',
    palette='viridis',
    diag_kind='kde',
    plot_kws={'alpha': 0.5, 's': 15},
    corner=True
)
g.figure.suptitle('Pair Plot of Key Features (coloured by Age Bin)', y=1.02,
plt.savefig('plots/pair_plot_of_key_features_coloured_by_age_bin.png', dpi=1
plt.show()
```

Pair Plot of Key Features (coloured by Age Bin)

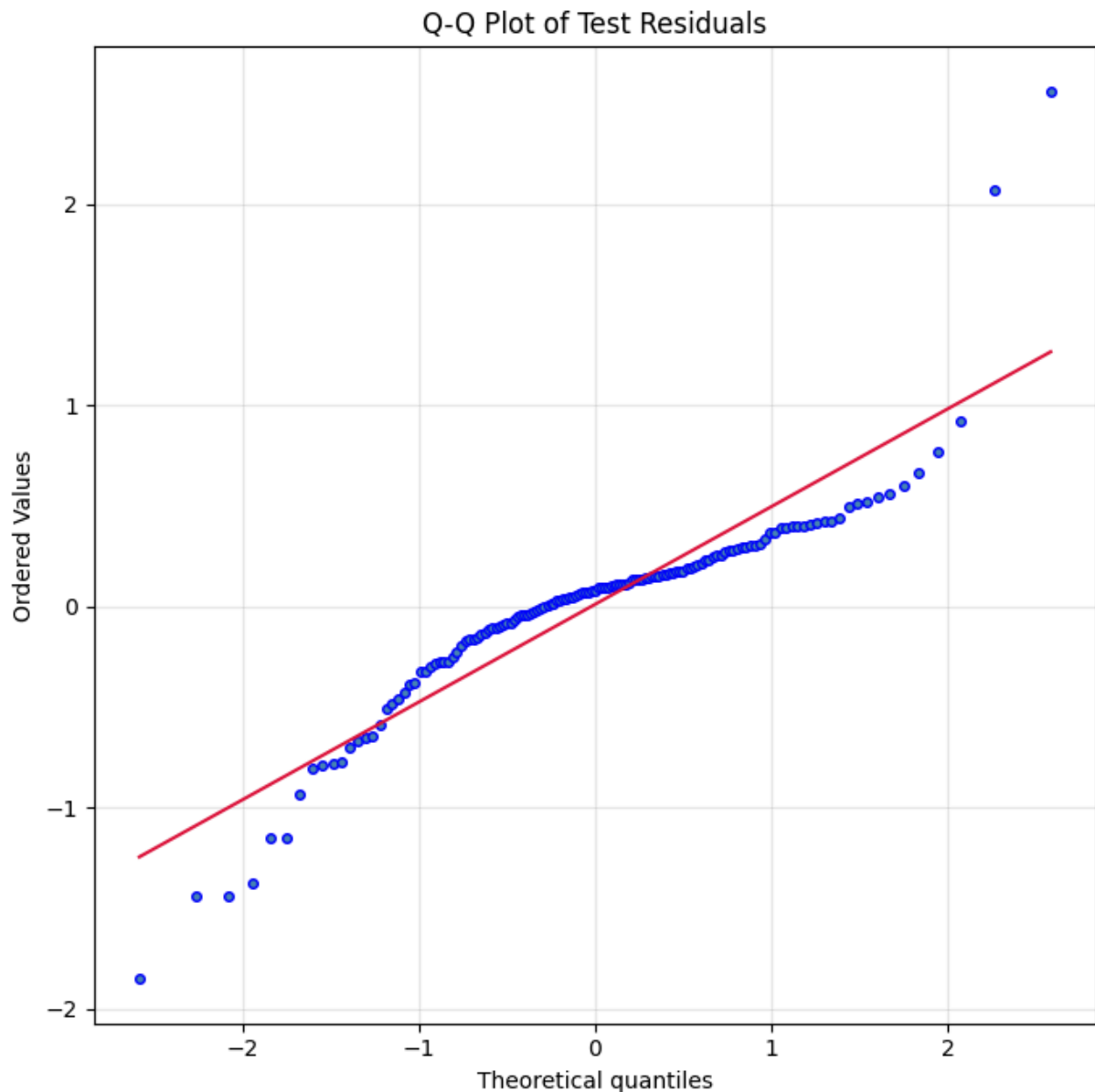


```
In [24]: # 3. KDE Overlay: True vs Predicted Age Distribution
plt.figure(figsize=(9, 6))
sns.kdeplot(y_test, label='True Age', fill=True, alpha=0.35, color='steelblue')
sns.kdeplot(y_pred, label='Predicted Age', fill=True, alpha=0.35, color='coral')
plt.xlabel('Age (Gyr)')
plt.ylabel('Density')
plt.title('Kernel Density Estimate: True vs Predicted Stellar Age')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('plots/kernel_density_estimate_true_vs_predicted_stellar_age.png')
plt.show()
```



```
In [25]: # 4. Q-Q Plot of Residuals (check for normality)
from scipy import stats

fig, ax = plt.subplots(figsize=(7, 7))
stats.probplot(residuals.values, dist='norm', plot=ax)
ax.set_title('Q-Q Plot of Test Residuals')
ax.get_lines()[0].set_markerfacecolor('steelblue')
ax.get_lines()[0].set_markersize(4)
ax.get_lines()[1].set_color('crimson')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('plots/q_q_plot_of_test_residuals.png', dpi=150, bbox_inches='tight')
plt.show()
```



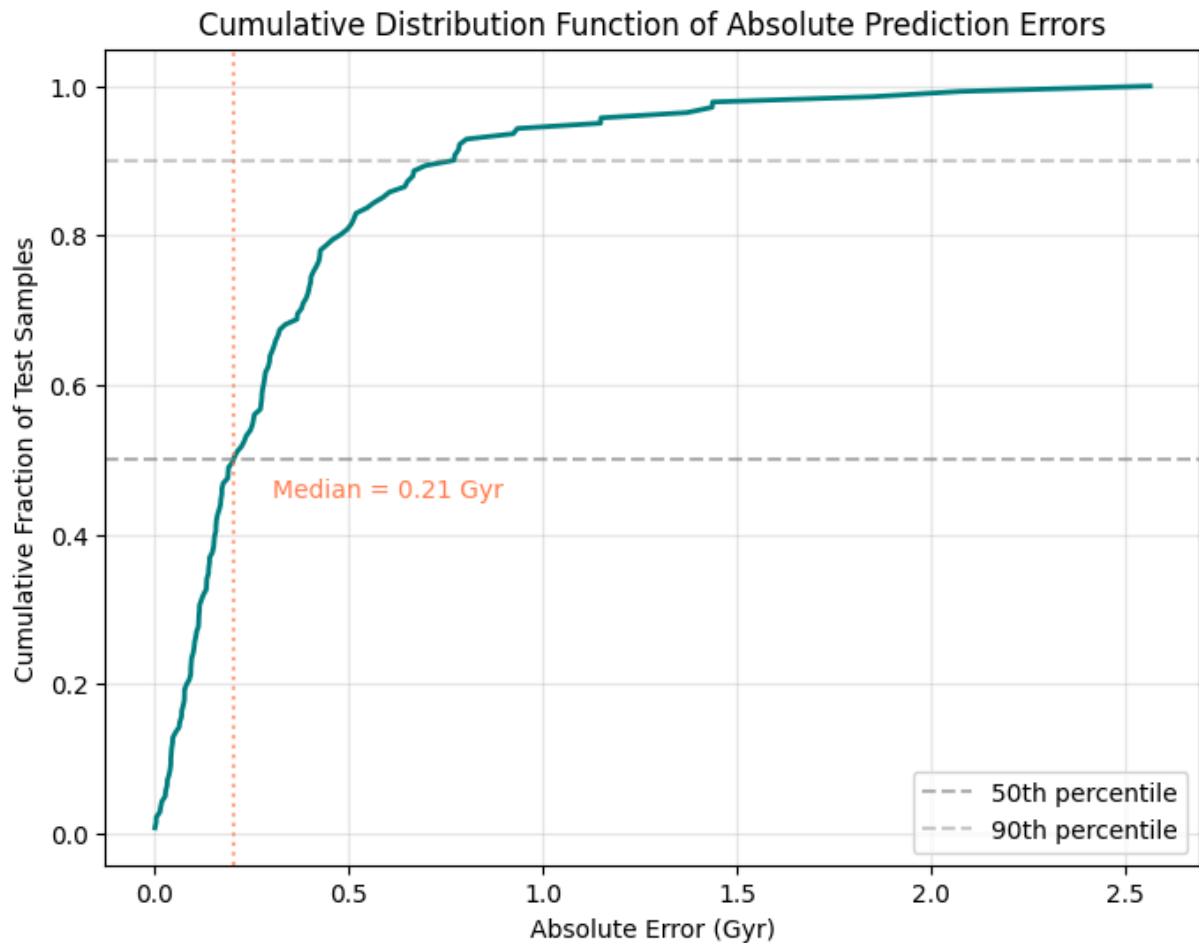
```
In [26]: # 5. Cumulative Distribution of Absolute Errors
abs_errors = np.abs(y_test.values - y_pred)
sorted_err = np.sort(abs_errors)
cdf = np.arange(1, len(sorted_err) + 1) / len(sorted_err)

plt.figure(figsize=(8, 6))
plt.plot(sorted_err, cdf, linewidth=2, color='teal')
plt.axhline(0.5, linestyle='--', color='grey', alpha=0.6, label='50th percer')
plt.axhline(0.9, linestyle='--', color='grey', alpha=0.4, label='90th percer')

# Annotate the median error
median_err = np.median(abs_errors)
plt.axvline(median_err, linestyle=':', color='coral', alpha=0.7)
plt.text(median_err + 0.1, 0.45, f'Median = {median_err:.2f} Gyr', fontsize=12)

plt.xlabel('Absolute Error (Gyr)')
plt.ylabel('Cumulative Fraction of Test Samples')
plt.title('Cumulative Distribution Function of Absolute Prediction Errors')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.savefig('plots/cumulative_distribution_function_of_absolute_prediction_e
plt.show()
```



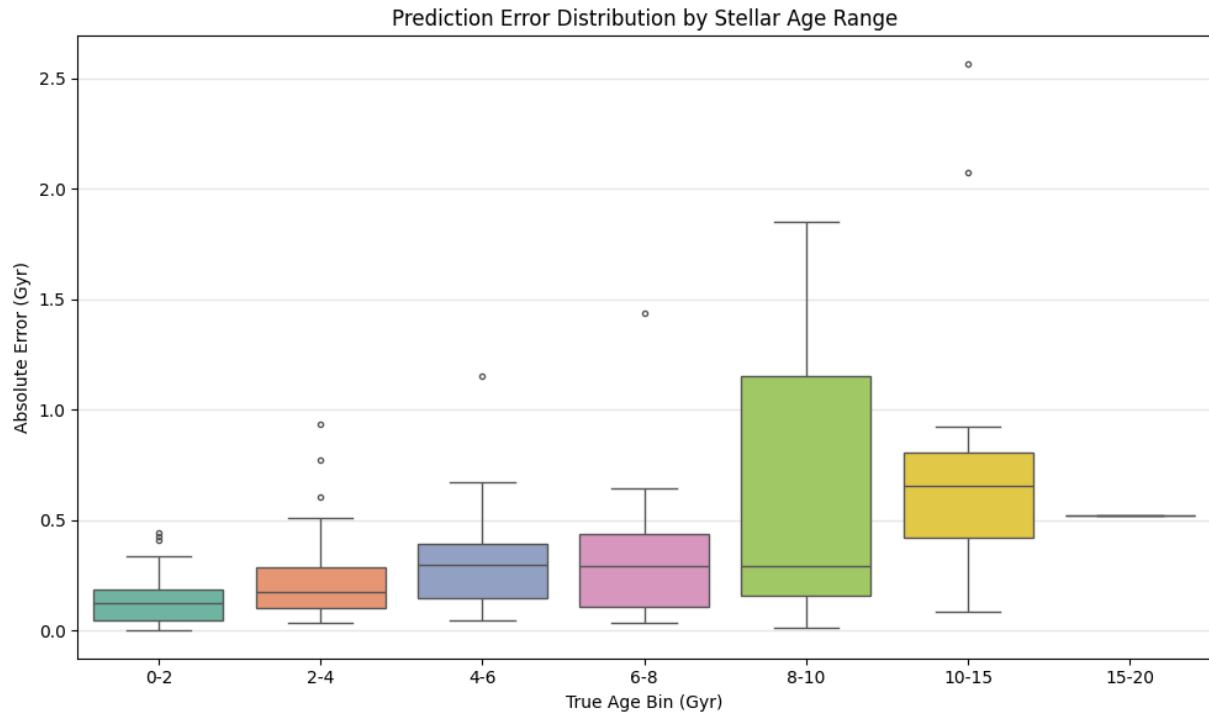
```
In [27]: # 6. Box Plot of Absolute Error grouped by True-Age Bins
test_results_tmp = pd.DataFrame({
    'true_age': y_test.values,
    'abs_error': np.abs(y_test.values - y_pred)
})
test_results_tmp['age_bin'] = pd.cut(
    test_results_tmp['true_age'],
    bins=[0, 2, 4, 6, 8, 10, 15, 20],
    labels=['0-2', '2-4', '4-6', '6-8', '8-10', '10-15', '15-20']
)

plt.figure(figsize=(10, 6))
sns.boxplot(x='age_bin', y='abs_error', data=test_results_tmp,
            palette='Set2', fliersize=3)
plt.xlabel('True Age Bin (Gyr)')
plt.ylabel('Absolute Error (Gyr)')
plt.title('Prediction Error Distribution by Stellar Age Range')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('plots/prediction_error_distribution_by_stellar_age_range.png',
plt.show()
```

```
/var/folders/1j/6168x29956dgb6w5h22437l00000gn/T/ipykernel_97820/2415867960.py:13: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x='age_bin', y='abs_error', data=test_results_tmp,
```



```
In [28]: # 7. Permutation Feature Importance
from sklearn.inspection import permutation_importance

perm_result = permutation_importance(
    baseline_model, X_test_scaled, y_test,
    n_repeats=15, random_state=42,
    scoring='neg_mean_squared_error'
)

sorted_idx = perm_result.importances_mean.argsort()

plt.figure(figsize=(10, 7))
plt.barh(
    [feature_cols[i] for i in sorted_idx],
    perm_result.importances_mean[sorted_idx],
    xerr=perm_result.importances_std[sorted_idx],
    color='teal', alpha=0.8
)

plt.xlabel('Mean Decrease in Neg-MSE')
plt.title('Permutation Feature Importance (Baseline Model)')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('plots/permutation_feature_importance_baseline_model.png', dpi=100)
plt.show()
```

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 4ms/step

5/5  0s 1ms/step
1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

Iteration	5/5	1/5	0s	1ms/step	4ms/step
1	5/5	1/5	0s	1ms/step	4ms/step
2	5/5	1/5	0s	1ms/step	4ms/step
3	5/5	1/5	0s	1ms/step	4ms/step
4	5/5	1/5	0s	1ms/step	4ms/step
5	5/5	1/5	0s	1ms/step	4ms/step
6	5/5	1/5	0s	1ms/step	4ms/step
7	5/5	1/5	0s	1ms/step	4ms/step
8	5/5	1/5	0s	1ms/step	4ms/step
9	5/5	1/5	0s	1ms/step	4ms/step
10	5/5	1/5	0s	1ms/step	4ms/step
11	5/5	1/5	0s	1ms/step	4ms/step
12	5/5	1/5	0s	1ms/step	4ms/step
13	5/5	1/5	0s	1ms/step	4ms/step
14	5/5	1/5	0s	1ms/step	4ms/step
15	5/5	1/5	0s	1ms/step	4ms/step
16	5/5	1/5	0s	1ms/step	4ms/step
17	5/5	1/5	0s	1ms/step	4ms/step
18	5/5	1/5	0s	1ms/step	4ms/step
19	5/5	1/5	0s	1ms/step	4ms/step
20	5/5	1/5	0s	1ms/step	4ms/step
21	5/5	1/5	0s	1ms/step	4ms/step
22	5/5	1/5	0s	1ms/step	4ms/step
23	5/5	1/5	0s	1ms/step	4ms/step
24	5/5	1/5	0s	1ms/step	4ms/step
25	5/5	1/5	0s	1ms/step	4ms/step
26	5/5	1/5	0s	1ms/step	4ms/step
27	5/5	1/5	0s	1ms/step	4ms/step
28	5/5	1/5	0s	1ms/step	4ms/step
29	5/5	1/5	0s	1ms/step	4ms/step
30	5/5	1/5	0s	1ms/step	4ms/step
31	5/5	1/5	0s	1ms/step	4ms/step
32	5/5	1/5	0s	1ms/step	4ms/step
33	5/5	1/5	0s	1ms/step	4ms/step
34	5/5	1/5	0s	1ms/step	4ms/step
35	5/5	1/5	0s	1ms/step	4ms/step
36	5/5	1/5	0s	1ms/step	4ms/step
37	5/5	1/5	0s	1ms/step	4ms/step
38	5/5	1/5	0s	1ms/step	4ms/step
39	5/5	1/5	0s	1ms/step	4ms/step
40	5/5	1/5	0s	1ms/step	4ms/step
41	5/5	1/5	0s	1ms/step	4ms/step
42	5/5	1/5	0s	1ms/step	4ms/step
43	5/5	1/5	0s	1ms/step	4ms/step
44	5/5	1/5	0s	1ms/step	4ms/step
45	5/5	1/5	0s	1ms/step	4ms/step
46	5/5	1/5	0s	1ms/step	4ms/step
47	5/5	1/5	0s	1ms/step	4ms/step
48	5/5	1/5	0s	1ms/step	4ms/step
49	5/5	1/5	0s	1ms/step	4ms/step
50	5/5	1/5	0s	1ms/step	4ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 3ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 3ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 3ms/step

5/5  0s 1ms/step
1/5   0s 3ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step
1/5   0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

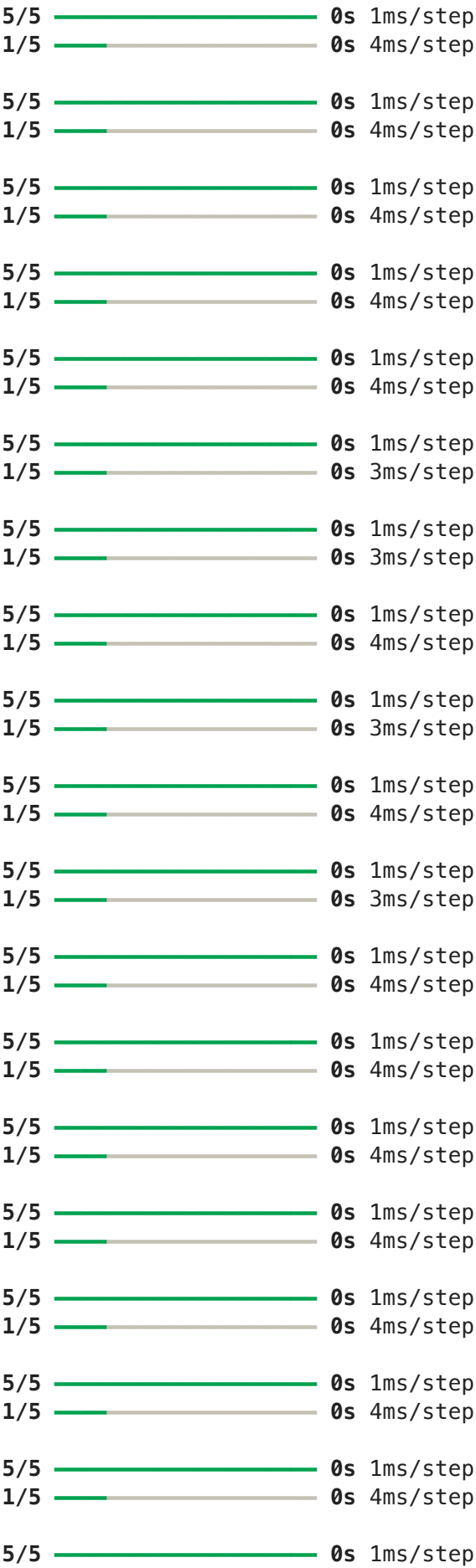
1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step



1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 3ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5  0s 1ms/step

1/5  0s 4ms/step

5/5 0s 1ms/step
1/5 0s 4ms/step

5/5 0s 1ms/step
1/5 0s 3ms/step

5/5 0s 1ms/step

